

メモリ領域の再利用と中間値の再計算による SQIsign の省メモリ実装

Hiro Nakanishi

独立研究者

quantumsity@protonmail.com

プレプリント版：2026 年 9 月 5 日

概要

SQIsign の公開鍵と署名は小さいが、鍵生成と署名に必要な作業メモリは小さくない。SQIsign v2.0.1 Level I の Compact 実装では、`find_uv` が同時に保持する五つの領域だけで 6,339,868 bytes を要し、RP2350 のオンチップ SRAM を超える。本稿では、同時に使わない領域を型付き `union` で共有する生存期間に基づくメモリ割当てと、候補ノルムのビット長と上位 64 bit だけを保持する方法を組み合わせる。保持したビット長と上位ビットが両方とも一致した場合は候補から全精度ノルムを再生成する。このため、衝突確率を仮定せずに元の比較順序を保存できる。

Level I/RADIX32 の操作用アリーナは 353,008 bytes から 172,080 bytes へ 51.2532% 減少した。v2 ファームウェアは GNU Multiple Precision Arithmetic Library (GMP)、動的メモリ確保、外部 PSRAM を用いない。予約する SRAM の合計は 321,408 bytes であり、211,072 bytes を未予約として残す。公式入力ベクトル 100 本を用いた差分適合試験では、同一コミットから構築した通常 API と低メモリ API の出力が一致した。この試験は、旧公式応答を期待値とする既知解テストではない。リンク済みの固定ファームウェアイメージを解析して得た鍵生成、署名、検証の Process Stack Pointer (PSP) 上界は、それぞれ 92,192、120,664、20,980 bytes である。これらの値は、最大 212 bytes の例外進入分を含み、131,072 byte の PSP 予約内に収まる。ただし、割込み処理、割込みの入れ子、および例外進入時の Main Stack Pointer (MSP) 使用量は含まない。

v3.0 では、同じ配置原理を二つの関数にある三組の領域へ適用した。公式版と適応版は、三つのパラメータ集合について合計 300 種類の入力を各一回処理した。二つの関数の Arm 用スタックフレームは、六つの比較すべてで減少した。RP2350 上で `p324_3` の第 0 ベクトルを処理したとき、署名時の PSP 使用量の実測値は 101,060 bytes から 96,396 bytes へ、4,664 bytes 減少した。この実機結果は、一つのパラメータ集合と一つの決定的実行経路に限られる。v2 と v3 の両方で、入力に応じて実行時間、制御フロー、メモリアドレスが変化することを検出した。したがって、定数時間性、電力・電磁波への耐性、および製品利用可能性は本稿の結論に含めない。

キーワード: 耐量子暗号、SQIsign、マイクロコントローラ、省メモリ実装、生存期間解析、サイドチャネル

1 はじめに

NIST の追加耐量子デジタル署名標準化では、SQIsign が第 3 ラウンド候補の一つに選ばれている [1, 2]。SQIsign は、超特異楕円曲線間の同種写像と四元数イデアルの対応を利用し、非常に小さな公開鍵と署名を実現する [3, 4, 5]。通信データが小さいことは、帯域、不揮発メモリ、証明書サイズが制約される機器にとって重要である。

しかし、通信形式が小さいことと、鍵生成・署名時の作業領域が小さいことは別の性質である。SQIsign の署名処理では、四元数算術、4 次元格子処理、ノルム方程式、候補探索、多倍長整数演算が重なる。v2 公式実装は GMP と動的確保を前提とする。固定精度整数版 [6] と Compact Quaternion Algorithms [7] は整数値の上界を与えたが、プロトコル全体で同時に使う配列の配置までは扱わない。本研究の出発点とした Compact 実装では、一回の `find_uv` が保持する五つの領域だけで 6,339,868 bytes を要する。これは RP2350 の全オンチップ SRAM 532,480 bytes の約 11.91 倍である。固定精度化によってヒープを除去しても、同じ領域を巨大なスタック配列または静的配列へ移すだけでは Cortex-M 上で実行できない。

組込み SQIsign については、マイクロコントローラ向け耐量子暗号評価基盤 pqm4 の調査が、2023 年の第 1 ラウンド提出実装について、GMP と動的確保を理由に SQIsign の統合を見送った [8]。後続研究も主として検証 (Verify) を対象としてきた [9, 10]。完全な v2 の鍵生成 (KeyGen)、署名 (Sign)、検証を扱う高速実装は、ホスト計算機または Armv8-A 環境を対象とする [11]。一方、2026 年 9 月 1 日に公開された v3.0 は、外部 GMP と浮動小数点ライブラリへの依存を除去し、Cortex-M4 向けの完全な KeyGen、Sign、Verify 実装を提供した [5, 12]。このため、v2 をオンチップ SRAM 内で実行できるかという問題と、再設計された v3 へ同じ配置原理を転用できるかという問題を分けて評価する。

本研究では暗号方式そのものを変更せず、値の表現、保持、再生成、および所有権を再設計する。具体的には、次の四つの研究課題を扱う。

- Q1. SQIsign v2.0.1 の一つの決定的 KeyGen、Sign、Verify 経路を、GMP、ヒープ、外部 PSRAM なしで Cortex-M 級のオンチップ SRAM 内に収められるか。
- Q2. 全精度の候補ノルム表を常駐させず、元実装と同じ比較順序と暗号出力を保てるか。
- Q3. RAM、スタック、フラッシュ、実行時間の間にどのようなトレードオフが生じるか。また、省メモリ化の前後にある入力依存性を区別できるか。
- Q4. v2 固有の候補探索が消えた SQIsign v3 でも、生存期間に基づく領域共有を別の大きなスタックフレームへ転用できるか。

本稿では、オブジェクトの生存期間が重ならない場合に、同じ物理メモリを再利用する。また、候補ノルムのビット長と上位 64 bit を保持し、両者が一致した場合には全精度値を再計算して比較する。この変更の直前にある静的アリーナ構成を**全精度表実装**、変更後の構成を *v2 提案実装* とする。v3 では、一つの関数に二組の領域共有を加えた実装と、二つの関数に合計三組の領域共有を加えた実装を区別する。以下で *v3 適応実装* とだけ記す場合は、2 関数を変更した実装を指す。

本稿の貢献は以下の通りである。

1. **プロトコル全体を対象とする生存期間に基づくメモリ割当て**。作業領域を、型、サイズ、アラインメント、生存期間、秘密性属性を持つオブジェクトとして表す。同時に使われない領域を型付きアリーナで共有する問題として定式化し、KeyGen、Sign、Verify が逐次利用する共有領域まで削減対象に含める。
2. **ビット長・上位ビットの比較と全精度ノルムの再計算**。候補ノルムのビット長と上位 64 bit を保持し、両者が同じ場合だけ候補から全精度ノルムを再生成する。比較器が元の全精度比較と同じ全順序を与えることを示し、意図的に保持した情報を衝突させる試験を含む差分試験で検証する。
3. **静的な Cortex-M 実装と RAM・スタック・フラッシュ・実行時間のトレードオフ評価**。SQIsign v2.0.1 Level I/RADIX32 について、GMP、動的メモリ確保関数、可変長配列 (VLA) を、KeyGen、Sign、Verify から到達するリンク後のコードから除去した。v2 提案実装は操作用アリーナを 353,008 bytes から 172,080 bytes へ 51.2532% 削減し、SRAM 予約合計を 321,408 bytes、未予約 SRAM を 211,072 bytes とする。フラッシュ使用量は 880 bytes 増加する。12 個の固定入力、公式入力ベクトル 100 本による通常・低メモリ API 差分適合試験、固定ファームウェアイメージの PSP 解析、および実機測定により、この構成を検証した。
4. **SQIsign v3 への適用評価**。公式 v3.0 と 2 関数を変更した実装を別々に構築し、同じ基準版とコンパイル条件で比較する。2 関数を変更した実装は、`quat_111_dual_reduce_ideal` と `protocols_sign` にある計三組の領域を共有する。三つのパラメータ集合すべてでホスト機能試験と Arm 用スタックフレームの監査を行う。`p324_3` の実機試験では、Sign の PSP 使用量の実測値を 4,664 bytes 削減する。v2 固有のビット長と上位ビットの組は転用せず、版を越えて転用できた配置原理と v2 固有の変更を区別する。
5. **実装安全性の評価**。固定入力とランダム入力の実行時間検査、署名処理全体の制御フロー・メモリアドレス差分、および既報の単純電力解析 (SPA) [13] との実行経路の対応を評価する。現実装に入力依存性が残るという検出結果を報告し、省メモリ化を定数時間化または電力・電磁波耐性として主張しない。

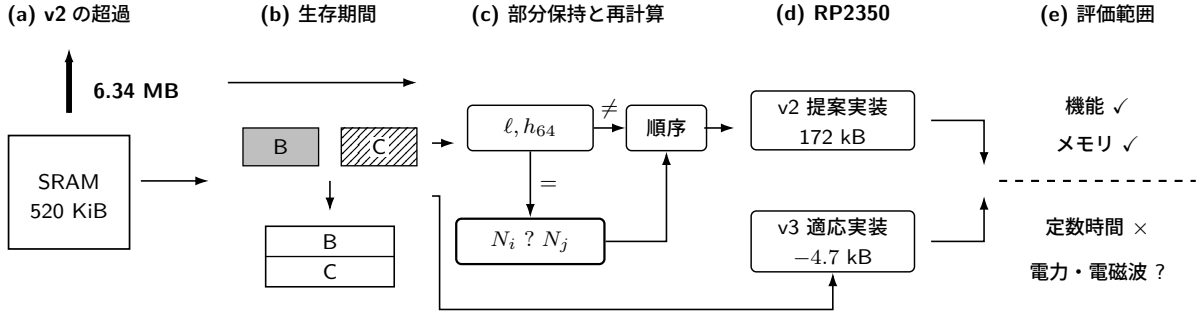


図1 本研究の全体像。v2 提案実装では生存期間に基づくメモリ割当てと、ノルムの部分保持および再計算を組み合わせる。v3 適応実装では、生存期間に基づくメモリ割当てを格子簡約と署名処理の二つの関数へ転用する。両実装を RP2350 上で評価し、機能・メモリの評価とサイドチャネル評価を分けて示す。

2 関連研究

生存期間が重ならない領域を共有する原理自体は新しくない。同時に生存する値を干渉辺で表す考え方は、グラフ彩色に基づくレジスタ割当てで確立されている [14]。異なる大きさの配列や構造体を対象とするコンパイル時メモリ割当ても研究されている [15]。したがって、本稿は領域共有そのものの発明を主張しない。本稿の対象は、SQIsign の完全な KeyGen、Sign、Verify について、分岐、再試行、失敗経路、型、アラインメント、および秘密消去を含む生存期間の条件を定めることである。この条件を、必要時の全精度再生成による常駐データの削減と組み合わせ、リンク済み実行ファイル (ELF) と実機測定から総 SRAM 予約量を評価する。

固定精度整数版 SQIsign は、四元数中間値について Level I/III/V で 7026/10713/14150 bit の一様上界を与え、GMP を使わない実装可能性を示した [6]。Compact Quaternion Algorithms の現行改訂は、この上界を 1665/2521/3319 bit へ縮小し、ホスト計算機上の鍵生成・署名を大幅に高速化した [7]。本稿の v2 実装も Level I で 1665 bit の絶対値上界を用いる。本研究はこの値幅を出発点とするが、同時に使われる候補表とプロトコル全体の領域配置を主対象とする。

Qlapoti は、四元数イデアルを同種写像へ変換する IdealToIsogeny 処理を改善し、SQIsign の実行時間とヒープ使用量を大きく削減した [16, 17]。その後の Qlapoty は、公開済み Qlapoti の記述と実装の差異、および失敗確率解析を再検討し、新しいノルム方程式アルゴリズムと SQIsign 署名の追加高速化を報告した [18]。これらの成果は、本稿の出発点となった実装に含まれる。一方、既報のヒープ使用量やホスト計算機上の実行時間は、スタック、割込み用予約、書換え可能な大域領域を含む Cortex-M の SRAM 予約量の合計ではない。したがって、v2 提案実装との同一条件の比較には使えない。

pqm4 による追加署名候補の調査では、SQIsign は GMP と動的確保のため組込み統合から除外された [8]。その後、Cortex-M4 向け高速有限体演算の生成により v2 Level I の Verify が高速化されたが、KeyGen と Sign は対象外である [9]。一次元 SQIsign の Cortex-M4 実装も主な実機対象は Verify である [10]。一方、32-bit Cortex-A/Linux 実装、Armv8-A NEON 実装、および AVX-512 実装は完全な KeyGen、Sign、Verify を扱うが、Cortex-M 級のオンチップ SRAM だけを対象としない [19, 11, 20]。純 Rust の sqisign-rs は GMP を用いない完全実装の近接例だが、署名処理は alloc と num-bigint を使い、Cortex-M 上の全操作のメモリ量を報告しない [21]。以上から、最も近い先行例を「全操作を実装するがホスト計算機向け」「GMP を使わないがヒープを使用」「Cortex-M 向けだが Verify のみ」の三群に分けて比較する。

公式 v3 実装は、この状況を変えた。v3 は独自の固定精度整数層を採用し、Cortex-M4 上の完全な KeyGen、Sign、Verify を提供する。Level I における各操作のスタック使用量は 56、99、37 kB と報告されている [5, 12]。したがって、本稿は、Cortex-M 上で初めて全操作を実行したとは主張しない。公式 v3 を無改変の対照実装とし、同じソースコードのコミットと構築条件の下で、生存期間に基づくメモリ割当てが署名時のスタックを削減するかを調べる。

安全性については、整数算術 [22]、4 次元格子簡約 [23]、四元数処理 [24] などの定数時間化が進んでいる。しか

し、いずれも現在の低メモリ署名実装全体を定数時間化するものではない。既報の単純電力解析 [13] は、この不足が実際の攻撃経路につながることを示している。

表 1 本研究に直接関係する実装研究の位置付け

研究	KeyGen・Sign・Verify	対象	本稿との差
Official v2[4]	ホストでは有	x86/GMP	ヒープ・GMP 前提
Official v3[5]	有	Cortex-M4 を含む	固定精度化済み。本稿 v3 評価の対照実装
Fixed precision[6]	ホスト	x86	値幅を証明、全同時生存 SRAM は未報告
Compact quaternion[7]	ホスト	x86	本稿の出発点、公開実装の候補表が MCU SRAM 超過
Qlapoti/Qlapoty[16, 18]	ホストの Sign 経路	x86/GMP	アルゴリズム・ヒープ・速度を改善、スタックを含む MCU 実測ではない
pqm4 調査 [8]	組込み統合から除外	Cortex-M4	GMP・動的確保が障壁
m4-modarith[9]	Verify のみ	Cortex-M4	現行 v2 の組込み K/S なし
sqisign-rs[21]	有	ホスト/no_std	非 GMP だが Sign はヒープ使用
32-bit/AVX-512[19, 20]	有	Cortex-A/x86	ホスト級のメモリ環境
SQIsign on ARM[11]	有	ArmV8-A/NEON	Cortex-M 級でなく GMP を保持
本研究	有	RP2350/M33	v2 は全操作の静的領域を計上、v3 は局所スタック領域を共有

2026 年 9 月 4 日に v3 公開後の再監査を行った。v2.0.1 Level I の実 KeyGen、Sign、Verify、Cortex-M 級実機、オンチップ SRAM のみ、GMP なし、動的確保なし、ファームウェア全体の SRAM 予約合計 520 KiB 未満を同時に示す公開例は見つからなかった。この否定的検索結果は v2 だけに限定し、「世界初」の証明とは扱わない。公式 v3 は既に Cortex-M4 の完全な KeyGen、Sign、Verify を提供するため、v3 について同じ不在主張を行わない。検索対象、検索式、除外基準、および上記反例群は日付付き検索ログ <https://github.com/quantumshiro/tinysqisign/blob/main/results/literature/related-work-search-2026-09-04.md> に固定した。

3 背景と問題設定

3.1 対象方式と実行環境

SQIsign は、超特異楕円曲線の自己準同型環と四元数代数の極大整環の対応を計算に用いる署名方式である [3]。本研究が主対象とする v2.0.1 は、署名の応答計算で楕円曲線の積、すなわち二次元アーベル多様体上の $(2, 2)$ 同種写像鎖を用いる第 2 ラウンド仕様系列である [4]。したがって、本稿の v2 実験は一次元 SQIsign 実装を対象としない。旧第 1 ラウンド実装や一次元検証実装のメモリ・性能値は、そのまま比較対象にできない。v2 提案実装は Level I の完全な KeyGen、Sign、Verify 経路を対象とする。

計算の主要部分は、有限体・楕円曲線演算、四元数イデアル演算、4 次元格子処理、および固定精度多倍長整数演算からなる。有限体側はコンパイル時に幅が定まる。一方、四元数側では大きな中間整数、候補列、ソート用情報、格子状態が同時に使われる場合がある。固定精度化は動的な整数長を排除するが、全ルーチンへ一様な最大幅を適用すると、大きな配列がスタックまたは静的領域へ移るだけである。したがって、固定精度化だけで総 SRAM が小さくなるとは限らない。

SQIsign v3.0 は 2026 年 9 月 1 日に公開された第 3 ラウンド仕様である [5]。Level I に相当する p324_3 では、公開鍵、秘密鍵、署名がそれぞれ 83、270、200 bytes である。v3 は、新しい攻撃評価に対応してパラメータを拡大した。また、四元数イデアルの慣性表現、整数増大に対するより小さい上界、新しい格子簡約、イデアルから同種写像への変換、および新しい応答分布を導入した。外部 GMP と DPE 浮動小数点ライブラリは、独自の固定精度整数実装と浮動小数点を使わない処理へ置き換えられた。公式ソースコードは x86_64、Arm64、Cortex-M4 を対象とし、Cortex-M4 上の完全な KeyGen、Sign、Verify について性能とスタック使用量を報告する [5, 12]。

v3 は v2 への小さな修正ではない。v2 提案実装で最大の削減を生んだ find_uv と全精度候補ノルム表は、公式 v3 ソースコードのコミット 6d017708db403bf83977fa70770fc4f7f9e9ff21 から到達する処理に存在しない [12]。したがって、v2 のビット長と上位ビットの組には v3 上の適用対象がない。一方、生存期間が重ならない大きな局所状態は v3 にもあり、生存期間に基づくメモリ割当てを適用できる。本稿は、v2 固有のデータ縮約と、版を越えて使える配置原理を分けて評価する。なお、v3 仕様は、応答分布が変更されたため、v2 と v3 の安全性仮定を技術的に直接比較できないと説明している [5]。本稿も暗号学的安全性の版間順位を主張しない。

対象は Raspberry Pi Pico 2 に搭載された RP2350 である。実験では、単精度浮動小数点演算器とセキュリティ拡張機能を備える Arm Cortex-M33[25] を 1 コア使用した。システムクロックは 150 MHz とし、532,480 bytes のオンチップ SRAM だけを使用した。外部 PSRAM は使用せず、コードおよび読み出し専用データは、基板上の外付けフラッシュから直接実行・参照する。この実行方式を execute in place (XIP) と呼ぶ。測定対象のファームウェアは Pico SDK 2.3.0、Arm GNU Toolchain 15.2.1、Release 構成で構築した。

RP2350 は一般的な Cortex-M4 評価ボードより SRAM が大きい、元の候補探索用領域はそれでも桁違いに大きい。また、割込み用 MSP、アプリケーション用 PSP、操作用共有領域、および実行時データを同じ物理 SRAM 内に置く必要がある。このため、本稿はヒープ使用量だけを評価しない。リンク時に確保する静的領域、スタック予約、および操作用アリーナを合計した SRAM 予約量の合計を評価指標とする。

機能・メモリ評価では、境界外アクセス、作業領域の前後に置いた保護領域の破壊、秘密値の未消去、動的メモリ確保関数または GMP への意図しない依存、および参照実装との出力不一致を失敗とする。物理攻撃者については、署名中の実行時間、制御フロー、メモリアドレス、消費電力、電磁波を観測できる強い実装攻撃者を想定する。評価対象は、ホスト計算機上の実行時間・構造トレース、RP2350 上の局所処理時間、および v3 の署名処理全体の実行時間である。署名全体の電力・電磁波波形は含まない。

本稿の機能・メモリ評価は、対象経路の出力一致と、定義したメモリ境界への適合を扱う。秘密非依存の実行、マスキング、フォールト耐性、製品品質の乱数生成はこの評価から分離し、実装安全性に関する限界として扱う。

3.2 同時生存メモリの最小化と領域共有の条件

操作中使用するオブジェクト集合を $O = \{o_1, \dots, o_n\}$ とする。入力、再試行、失敗終了を含む許容実行トレースの集合を $\mathcal{T}$ とする。各 o_i は、サイズ $s_i \in \mathbb{N}_{>0}$ 、アラインメント $a_i \in \mathbb{N}_{>0}$ 、型 τ_i 、秘密性属性 σ_i を持つ。トレース $t \in \mathcal{T}$ において o_i が生存する時点の集合を $L_i(t)$ とし、そのトレースに現れない場合は $L_i(t) = \emptyset$ とする。干渉辺を

$$(o_i, o_j) \in E \iff \exists t \in \mathcal{T} : L_i(t) \cap L_j(t) \neq \emptyset \quad (1)$$

で定義し、このグラフを $\mathcal{G} = (O, E)$ と書く。すなわち、どれか一つの許容経路で同時生存し得る二領域は干渉する。単一の正常系トレースで非交差だったことだけでは、重量を許さない。

アリーナ内のオフセットを $x_i \in \mathbb{Z}_{\geq 0}$ とすると、配置は次を満たさなければならない。

$$x_i \equiv 0 \pmod{a_i}, \quad (2)$$

$$(o_i, o_j) \in E \implies (x_i + s_i \leq x_j) \vee (x_j + s_j \leq x_i). \quad (3)$$

前後の保護領域などを除くアリーナ本体の使用量は

$$M_{\text{arena}} = \max_i (x_i + s_i) \quad (4)$$

である。最小化問題は、 M_{arena} を目的関数とする、アラインメント制約付きの記憶領域割当て問題になる。ただし、v2 提案実装は一般的な最適配置を求めるものではない。人手で定めた処理段階から実行可能な配置を C の struct と union で構成し、sizeof、offsetof、_Static_assert で検査する。アリーナの先頭アドレスも、各要素が要求するアラインメントを満たすものとする。実装ではバイト配列を任意の型へ変換しない。型付き union のアラインメント処理をコンパイラに委ね、アプリケーションバイナリインタフェース (ABI) の検査プログラムで確認する。

M_{arena} はファームウェア全体の SRAM 量ではない。アリーナ前後の保護領域を $g_{\text{pre}}, g_{\text{post}}$ 、PSP と MSP の予約を $R_{\text{PSP}}, R_{\text{MSP}}$ 、操作用共有領域とスタック以外の書換え可能な静的領域を R_{other} とする。このとき、ファームウェア全体で、各領域を重複して数えずに求めた SRAM 予約量の合計は

$$R_{\text{excl}} = (M_{\text{arena}} + g_{\text{pre}} + g_{\text{post}}) + R_{\text{PSP}} + R_{\text{MSP}} + R_{\text{other}}. \quad (5)$$

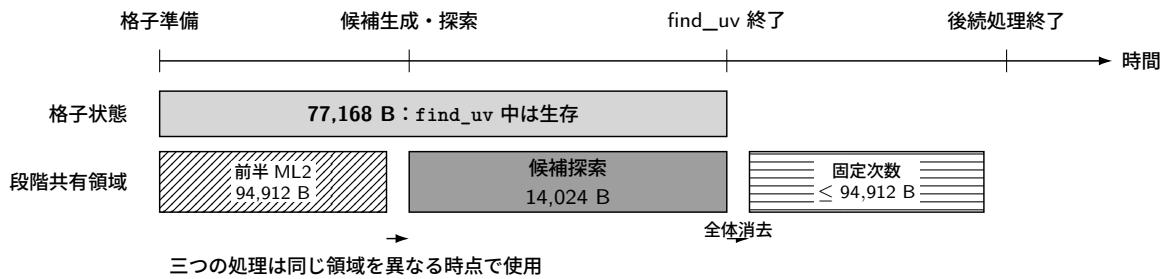
v2 提案実装では、 $M_{\text{arena}} = 172,080$ 、前後の保護領域の合計 128、 $R_{\text{PSP}} = 131,072$ 、 $R_{\text{MSP}} = 8,192$ 、 $R_{\text{other}} = 9,936$ であり、 $R_{\text{excl}} = 321,408$ bytes となる。フラッシュから直接実行する読み出し専用領域と未予約 SRAM は、この式に加えない。

領域の重畳は、単に「同時には使わないはず」という経験則では認めない。二つの型付き領域を同一 offset へ配置する条件を次のように定めた。

1. 呼出しグラフと処理段階の状態遷移において、生存期間が重ならない。
2. 呼出し先がポインタを現在の処理段階より後まで保存せず、返り値も当該領域を参照しない。
3. 次の処理段階へ移る前に秘密値を消去し、必要な箇所では領域全体が 0 であることを検査する。
4. C の別名参照規則とアラインメントを、型付き union の要素または検査済みのアクセス関数で保証する。バイト配列を無制限に別の型へ変換しない。
5. 失敗経路を含むすべての終了経路で、前後の保護領域、秘密値の消去、および未確定値が外部出力へ書き込まれていないことを検査する。

実装では、この条件を ABI 上のサイズ検査、`-Wvla -Walloca -Werror`、AddressSanitizer (ASan)、UndefinedBehaviorSanitizer (UBSan)、カナリア値、全領域の消去試験、および ELF シンボル監査で検査した。これらは完全な静的証明ではないが、人手で記述した生存期間の注釈とは異なる方法で誤りを検出する。

(a) 論理生存区間



(b) 物理配置



図 2 IdealToIsogeny 経路で共有する v2 提案実装の作業領域について、論理的な生存期間と物理配置を示す。格子状態は `find_uv` の実行中を通して生存するため、候補探索などの処理段階用領域とは共有できない。`find_uv` の共通出口で全体を消去した後は、同じ領域を固定次数処理が再利用する。横方向は処理順を表し、実行時間の比率を表さない。

本実装では、KeyGen、Sign、Verify を同時に実行しない。三操作の作業領域を一つの型付き union で共有し、各操作の終了時に使用した領域と共有アリーナを消去する。これにより、三操作に必要な SRAM の合計ではなく、最大値だけを確保すればよい。一方、この API は再入可能ではなく、並行署名や複数コアからの同時利用を許さない。1 コアで三操作を逐次実行することを API の前提とする。

4 提案手法

生存期間に基づく領域共有と、候補値を必要時に再生成する表現を組み合わせる。v2 では両者を用いて常駐状態を削減し、v3 では領域共有を二つの関数へ適用する。配置を C の型と注釈に対応付けることで、どの値が同時に存在するかを実装上でも追跡できるようにする。

4.1 常駐状態の削減

出発点の Compact 実装では、候補ベクトル、全ノルム、商、行数、ソート用記録を一回の `find_uv` で同時に保持していた。本研究では、各変換後に出力一致を確認しながら、以下の変更を順に適用した。

1. 値域が $[-2, 2]$ と証明される 4 座標候補を、広い多倍長表現から `int8_t[4]` へ圧縮する。
2. ソート用記録に候補を複製せず、`uint16_t` の索引配列をソートする。
3. 候補ごとの商を常駐させず、使用地点で決定的に再計算する。
4. 全行を保持する処理を、二行ずつ処理する方式へ変換し、確定した行の領域を再利用する。
5. 大きな可変長配列を呼出し側所有の作業領域へ移し、対象とする Arm 用コードの動的スタックフレームを 0 件にする。
6. ML2、候補探索、theta 鎖、離散対数、一括逆元計算、および多倍長整数の一時領域を処理段階ごとに共有する。
7. KeyGen、Sign、Verify の作業領域を一つの逐次利用領域へ統合する。
8. 最後に、全精度ノルム行をビット長と上位ビットの組と必要時の全精度再生成で置き換える。

前半の候補圧縮、索引化、二行処理は常駐データそのものを減らす。後半のアリーナ化は、同じ物理領域を異なる時点で再利用する。再計算はフラッシュ使用量と実行時間を増やす可能性があるため、各変換は RAM、スタック、フラッシュ、実行時間の四指標で評価する。

表 2 は、各変換の内容と計上範囲を示す。「二行ストリーミング」までの値は `find_uv` の候補探索用領域である。それ以降は、格子状態を含む呼出し側所有の作業領域または操作用アリーナの `sizeof` である。いずれもファームウェア全体の最大使用量ではない。「格子状態の明示所有」では 77,168 bytes の可変長配列を作業領域へ移すため、表の値が増える。しかし、同時に使うメモリの総量は変わらない。保存場所を移しただけの変更をメモリ削減に数えないため、この段階を表に残した。

表 2 主要な変換後に明示的に確保する領域の大きさ。計上範囲が異なる行同士を、ファームウェア全体の最大使用量として比較していない。

構成	明示領域 [B]	その構成で導入した変換
Compact 出発点	6,339,868	同時に確保する五つのヒープ領域
候補座標の圧縮	2,044,252	4-byte 候補座標と圧縮したソート用記録
索引ソート	1,905,728	候補の複製を廃し、 <code>uint16_t</code> 索引配列を再利用
商の遅延再計算	962,240	候補ごとの商を常駐させず、使用時に再計算
二行ストリーミング	275,840	全 7 行の常駐を二行へ変更
格子状態の明示所有	353,008	77,168 byte の格子用可変長配列を型付き領域へ移動（同時使用量は不変）
全精度表実装	353,008	ML2、theta、離散対数等の領域を共有し、対象 Arm コードの動的フレームを除去
v2 提案実装	172,080	全精度ノルム二行をビット長と上位ビットの組と再生成へ置換し、ML2 領域を共有

4.2 ビット長・上位ビットの比較と全精度ノルムの再計算

候補 v のノルム $N(v)$ は、保持済みの 4 座標圧縮ベクトルと、その行の Gram 行列 G_j および調整ノルム d_j から、二次形式を評価して再生成できる。この性質を使い、各候補の全精度整数 $N(v)$ を常駐させる代わりに、以下で定めるビット長と上位ビットだけを保持する。 G_j, d_j は秘密に由来する状態を含む可能性がある。したがって、ノルムを再生成できることはサイドチャネル安全性の根拠にならない。

定義 1 (ビット長と上位 w bit). 非負整数 n に対して、ビット長 $\ell(n)$ を $n = 0$ なら 0、 $n > 0$ なら $\lfloor \log_2 n \rfloor + 1$ とする。 $w = 64$ として、

$$h_w(n) = \begin{cases} n, & \ell(n) \leq w, \\ \lfloor n/2^{\ell(n)-w} \rfloor, & \ell(n) > w \end{cases} \quad (6)$$

を定める。組 $S_w(n) = (\ell(n), h_w(n))$ は n のビット長と上位ビットを保持する。

比較器は、(1) ビット長、(2) 上位 64 bit、(3) 保持した情報が同じ場合に再生成する全精度ノルム、(4) 元の列挙番号、の順に比較する。

命題 1 (順序保存). 各候補 v_i から $N(v_i)$ を全精度で再生成できるとする。上記比較器が与える候補の全順序は、組 $(N(v_i), i)$ を全精度整数比較と列挙番号の比較で辞書式順序に並べた結果と一致する。

証明. $\ell(N(v_i)) \neq \ell(N(v_j))$ なら、ビット長の小さい非負整数が必ず小さい。ビット長が等しく、 h_{64} が異なる場合、最上位から最初に異なる bit は保持された 64 bit 内に存在するため、その大小は全整数の大小と一致する。両者が等しい場合、保持した情報だけでは大小を決めず、二つのノルムを全精度で再生成して比較する。ノルムも等しい場合だけ列挙番号を比較する。したがって、すべての場合で $(N(v_i), i)$ の辞書式順序と一致する。□

この方法は、保持したビット長と上位ビットの一致が十分に起こりにくいとは仮定しない。例えば 2^{100} と $2^{100} + 1$ はビット長と上位 64 bit が一致するが、必ず全精度比較へ進み、正しい順序を返す。したがって、暗号学的ハッシュの衝突確率や、有限の入力集合で観測した衝突頻度は正しさの仮定に含まれない。

アルゴリズム 1 に、C 実装の `finduv_eval_norm`、`finduv_set_norm_sketch`、`finduv_compare_indices` に対応する処理を示す。FULLNORM は二次形式値を d_j で割り、余りが 0 かつ商が正であることまで検査する。したがって丸めた近似値を同値時の比較に使用しない。

ソート時は、二つの保持した情報が同じになるたびに両方のノルムを一時領域へ再生成し、比較直後に消去する。探索時は、ソート済みの索引が指す候補から必要なノルムをその場で再生成する。初期測定では、一回の `find_uv` あたり約 7,800 回のソート比較が発生した。メモリ・機能評価用の入力集合では、探索時の再評価回数は KeyGen で 2–30 回、Sign で 2–578 回であった。これとは別の固定鍵実行時間検査では Sign を 128 回実行し、そのうち 64 回にランダム入力を用いた。この試験では、2,062,361 回の比較中 5,041 回、すなわち 0.2444% で保持した情報が一致した。ランダム入力における探索時の再評価回数は 4–1677 回であった。2–578 回と 4–1677 回は異なる入力集合から得た値である。いずれも性能を説明する観測値であり、全入力に対する最悪回数の上界ではない。

一行のソートでは、 $m \leq 624$ 個の列挙番号を持つ `uint16_t` 索引配列を、追加配列を使わないヒープソートで整列する。比較回数は $O(m \log m)$ 、補助領域は $O(1)$ であり、再帰や標準 C ライブラリのソート用作業領域を使用しない。C の比較関数はエラー状態を直接返せない。全精度再生成に失敗した場合は `fatal` 状態を記録し、その比較だけは列挙番号順を返す。ヒープソート終了後に状態を検査して行全体を拒否するため、この暫定順を正常な出力として外部へ渡さない。

アルゴリズム 2 to 4 は、一行の生成・整列、二行の候補対探索、および二行だけを常駐させながら元の三角形順 (j_1, j_2) を保つ処理順を示す。ADMISSIBLE は、元実装と同じ順序で、符号の代表元を選び、全座標が 2 の倍数である候補または全座標が 3 の倍数である候補を除外し、該当する基底では位数 4 の対称性代表を選ぶ。Level I では $r \leq 7$ であり、一行の容量は 624 である。探索前に全行を検証するため、早い候補が見つかった場合でも後方にある不正な行を見逃さない。SEARCHPAIRS は、二つの候補ノルム n_1, n_2 に対する

$$un_1 + vn_2 = T, \quad u > 0, \quad v > 0 \quad (7)$$

の候補を元実装と同じ二重ループの順序で探す。具体的には、各合同解列について元実装と同じ $0 < v < \lfloor T/n_2 \rfloor$ を走査境界とする。本稿でいう「最初の解」は、全正整数解に関する新たな完全性主張ではなく、この参照走査領域と順序において最初に受理される解を意味する。v2 提案実装からの呼出しでは追加の二平方条件は無効である。探索中も n_1, n_2 を圧縮候補から再生成し、選ばれた候補だけを外部出力へ写す。この探索は可変回数であり、定数時間手続きではない。

アルゴリズム 1 必要な場合に全精度値へ戻るノルム比較

入力: 圧縮候補 v 、Gram 行列 G 、非零の調整ノルム d

出力: 成功時は正の全精度ノルム、10-byte 保持した情報、または $(N(v_i), i)$ の比較結果

```
1: function FULLNORM( $v, G, d$ )
2:    $p \leftarrow \text{WIDENTTOIBZ}(v)$ 
3:    $q \leftarrow \text{QUADRATICFORM}(p, G)$ 
4:    $(n, r) \leftarrow \text{DIVREM}(q, d)$ 
5:   if  $r \neq 0$  or  $n \leq 0$  then
6:     return FATAL
7:   end if
8:   return  $n$ 
9: end function
10: function MAKESKETCH( $n$ )
11:    $\ell \leftarrow \lfloor \log_2 n \rfloor + 1$ 
12:   if  $\ell > \text{UINT16\_MAX}$  then
13:     return FATAL
14:   end if
15:    $h \leftarrow n$  if  $\ell \leq 64$  else  $\lfloor n/2^{\ell-64} \rfloor$ 
16:   return  $(\ell, h)$ 
17: end function
18: function COMPARE( $i, j, V, S, G, d$ )
19:   if  $i = j$  then
20:     return 0
21:   else if  $S[i].\ell \neq S[j].\ell$  then
22:     return CMP( $S[i].\ell, S[j].\ell$ )
23:   else if  $S[i].h \neq S[j].h$  then
24:     return CMP( $S[i].h, S[j].h$ )
25:   end if
26:    $n_i \leftarrow \text{FULLNORM}(V[i], G, d)$ ;  $n_j \leftarrow \text{FULLNORM}(V[j], G, d)$ 
27:   if  $n_i = \text{FATAL}$  or  $n_j = \text{FATAL}$  then
28:     return FATAL
29:   else if  $n_i \neq n_j$  then
30:     return CMP( $n_i, n_j$ )
31:   else
32:     return CMP( $i, j$ )
33:   end if
34: end function
```

▷ $q = p^T G p$ を固定精度で評価

▷ ℓ : 16 bit、 h : 64 bit

▷ 元の列挙番号で全順序化

アルゴリズム 2 一行の候補生成と全精度順への整列

入力: 行 j 、常駐させる行の位置、呼出し側所有の作業領域 W

出力: 全精度キー $(N(v), i)$ で整列した圧縮候補行と個数 m 、または FATAL

```
1: function PREPAREROW( $j, \text{slot}, W$ )
2:    $m \leftarrow 0$ 
3:   for  $v \in [-2, 2]^4$  を元実装と同じ順序で走査 do
4:     if not ADMISSIBLE( $v, G_j$ ) then
5:       continue
6:     end if
7:      $n \leftarrow \text{FULLNORM}(v, G_j, d_j)$ 
8:     if  $n = \text{FATAL}$  then
9:       return FATAL
10:    end if
11:    if  $n \bmod 2 = 1$  then
12:      if  $m = 624$  then
13:        return FATAL
14:      end if
15:       $W.V[\text{slot}, m] \leftarrow \text{PACK4XINT8}(v)$ 
16:       $s \leftarrow \text{MAKE SKETCH}(n)$ 
17:      if  $s = \text{FATAL}$  then
18:        return FATAL
19:      end if
20:       $W.S[m] \leftarrow s; m \leftarrow m + 1$ 
21:    end if
22:  end for
23:   $P[k] \leftarrow k$  for  $0 \leq k < m$ 
24:   $\text{HEAPSORT}(P, \text{comparator COMPARE})$ 
25:  if 比較エラーを記録済み then
26:    return FATAL
27:  end if
28:  if not APPLYPERMUTATIONBYCYCLES( $W.V[\text{slot}], P$ ) then
29:    return FATAL
30:  end if
31:  return  $m$ 
32: end function
```

アルゴリズム 3 参照走査順を保つソート済み二行の候補対探索

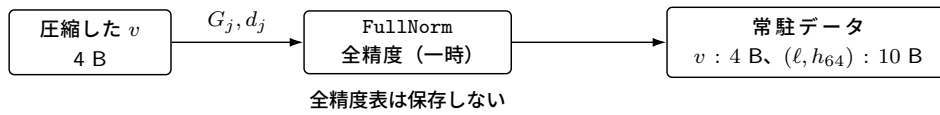
入力: 目標値 $T > 0$ 、ソート済み行 $A[0..m_1 - 1]$, $B[0..m_2 - 1]$ 、行記述 $R_k = (G_k, d_k)$ 、対角フラグ δ

出力: 参照実装の走査領域内で equation (7) を満たす最初の $(u, v, i_1, i_2, n_1, n_2)$ 、NOTFOUND、または FATAL

```

1: function SEARCHPAIRS( $T, A, B, R_1, R_2, \delta$ )
2:   for  $i_1 \leftarrow 0$  to  $m_1 - 1$  do
3:      $n_1 \leftarrow \text{FULLNORM}(A[i_1], R_1.G, R_1.d)$ 
4:     if  $n_1 = \text{FATAL}$  then
5:       return FATAL
6:     end if
7:      $a \leftarrow T \bmod n_1$ ;  $s \leftarrow i_1$  if  $\delta$  else 0
8:     for  $i_2 \leftarrow s$  to  $m_2 - 1$  do
9:       if  $\delta$  and  $i_1 = i_2$  then
10:         $n_2 \leftarrow n_1$ 
11:       else
12:         $n_2 \leftarrow \text{FULLNORM}(B[i_2], R_2.G, R_2.d)$ 
13:        if  $n_2 = \text{FATAL}$  then
14:          return FATAL
15:        end if
16:      end if
17:      if  $n_2^{-1} \bmod n_1$  が存在しない then
18:        continue
19:      end if
20:       $v \leftarrow a n_2^{-1} \bmod n_1$ ;  $q \leftarrow \lfloor T/n_2 \rfloor$ 
21:      while  $v < q$  do
22:         $(u, \rho) \leftarrow \text{DIVREM}(T - v n_2, n_1)$ 
23:        if  $\rho \neq 0$  or  $u \leq 0$  then
24:          return FATAL
25:        end if
26:        if  $v > 0$  then
27:          return  $(u, v, i_1, i_2, n_1, n_2)$ 
28:        end if
29:         $v \leftarrow v + n_1$ 
30:      end while
31:    end for
32:  end for
33:  return NOTFOUND
34: end function
  
```

(a) 生成時



(b) 比較時

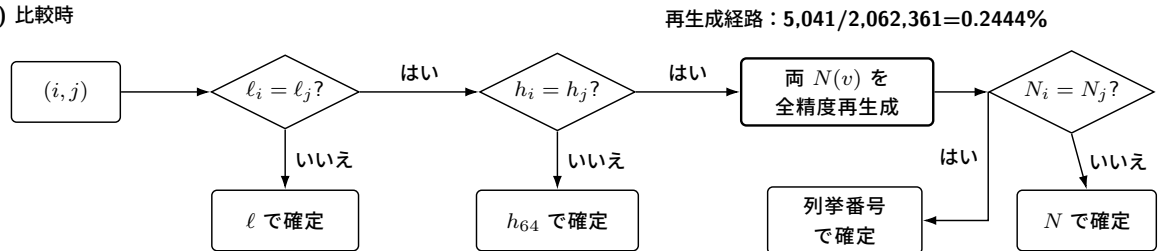


図3 ビット長と上位ビットの組の保存形式と比較経路。ビット長または上位 64 bit が異なる場合は、保持した情報だけで比較結果が決まる。両者が一致した場合は、圧縮候補から全精度ノルムを再生成する。図中の再生成率は固定鍵実行時間検査で観測した値であり、比較結果の正しさはこの頻度に依存しない。

アルゴリズム 4 全行を検証してから二行ずつ探索する v2 提案実装の `find_uv`

入力: 目標値 T 、基底イデアルと四元数環 $\mathcal{I}$ 、 r 個の整環、呼出し側所有の作業領域 W

出力: 元実装と同じ最初の解、NOTFOUND、または FATAL。すべての終了時に $W = 0$

```
1: function FINDUV-LM( $T, \mathcal{I}, W$ )
2:   if 引数または固定容量が不正 then
3:     SECURECLEAR( $W$ );
4:     return FATAL
5:   end if
6:   SECURECLEAR( $W$ )
7:    $L \leftarrow \text{PREPARELATTICESTATE}(T, \mathcal{I}, W.\text{phase.lattice\_mul})$ 
8:   if  $L = \text{FATAL}$  then
9:     return FINISH(FATAL,  $W$ )
10:  end if
11:   $R_j \leftarrow (L.\text{gram}[j], L.\text{adjusted\_norm}[j])$  for  $0 \leq j < r$ 
12:  SECURECLEAR( $W.\text{phase}$ )
13:  for  $j \leftarrow 0$  to  $r - 1$  do
14:     $c[j] \leftarrow \text{PREPAREROW}(j, 0, W)$ 
15:    if  $c[j] = \text{FATAL}$  then
16:      return FINISH(FATAL,  $W$ )
17:    end if
18:  end for
19:  for  $j_1 \leftarrow 0$  to  $r - 1$  do
20:     $m_1 \leftarrow \text{PREPAREROW}(j_1, 0, W)$ 
21:    if  $m_1 \neq c[j_1]$  then
22:      return FINISH(FATAL,  $W$ )
23:    end if
24:    for  $j_2 \leftarrow j_1$  to  $r - 1$  do
25:      if  $j_2 = j_1$  then
26:         $(V_2, m_2) \leftarrow (W.V[0], m_1)$ 
27:      else
28:         $m_2 \leftarrow \text{PREPAREROW}(j_2, 1, W)$ ;  $V_2 \leftarrow W.V[1]$ 
29:        if  $m_2 \neq c[j_2]$  then
30:          return FINISH(FATAL,  $W$ )
31:        end if
32:      end if
33:       $V_1 \leftarrow W.V[0][0..m_1 - 1]$ 
34:       $z \leftarrow \text{SEARCHPAIRS}(T, V_1, V_2, R_{j_1}, R_{j_2}, j_1 = j_2)$ 
35:      if  $z = \text{FATAL}$  then
36:        return FINISH(FATAL,  $W$ )
37:      end if
38:      if  $z$  が解である then
39:        if not VALIDATEANDPUBLISH( $z$ ) then
40:          return FINISH(FATAL,  $W$ )
41:        end if
42:        return FINISH(SUCCESS,  $W$ )
43:      end if
44:    end for
45:  end for
46:  return FINISH(NOTFOUND,  $W$ )
47: end function
```

▷ 候補処理へ一方向に遷移
▷ 全行を検証し、位置 0 を再利用

▷ 対角では同じ行を参照

命題 2 (最初の解の保存). 固定された入力に対し、参照実装と v2 提案実装の全精度演算がいずれも成功すると仮定する。両者が各行を $(N(v), i)$ 順に整列し、整環対、行内索引、合同解を同じ領域・順序で走査するならば、v2 提案実装は参照実装と同じ最初の解を返し、参照走査内に受理解がなければ両者とも NOTFOUND を返す。

証明. 命題 1 により、ビット長と上位ビットの組と全精度再生成で得る各行の順序は、全精度ノルム行の順序と一致する。最初の検証では個数以外の状態を探索へ持ち込まず、探索時には各行を同じ手順で再生成する。対角時の $i_2 = i_1$ 開始、非対角時の $i_2 = 0$ 開始、 (j_1, j_2) の三角形順、およびアルゴリズム 3 の合同解列はいずれも参照走査と同じである。したがって、最初に成功する組、またはすべての組の不成功が保存される。□

命題 2 の前提を実際の C 実装へ対応付けるため、全精度ノルム行を常駐させる全精度表実装（コミット 6b79cfb5...）と、その直後の v2 提案実装（コミット 71099e08...）の dim2id2iso.c を比較した。表 3 は、その対応を示す。検査スクリプトは両ソースコードのダイジェストを確認し、列挙、比較、走査、失敗処理に関する 22 個の条件を検査する。これにより、命題で仮定した走査領域と順序を、この二つのコミットにある C 実装へ対応付ける。

表 3 全精度表参照実装と v2 提案実装の find_uv 走査対応

義務	全精度表参照	v2 提案	失敗時
行生成	$x/y/z/w$ 、符号代表、2/3 除外、位数 4 代表、奇ノルム	同じループと判定の後に finduv_eval_norm	非整数商、非正值、容量超過は Fatal
ソートキー	全精度 $(N(v), i)$	bit 長、上位 64 bit、全精度再生成、 i	再生成失敗を記録し、行を Fatal にする
整環対	j_1 昇順、 $j_2 \geq j_1$	同じ三角形順、対角では同一行を参照	再生成数の不一致は Fatal
行内索引	i_1 昇順、対角 $i_2 = i_1$ 、非対角 $i_2 = 0$	find_uv_from_compact_lists で同順	ノルム再生成失敗は Fatal
合同解列	$v = (T \bmod n_1)n_2^{-1} \bmod n_1$ 、 $v \neq n_1$	同じ代表、増分、 $v < \lfloor T/n_2 \rfloor$	非可逆なら次候補へ進み、破損算術なら Fatal
出力・終了	成功後だけイデアルを検証して出力	同じ出力順。追加条件は本経路で無効	Success/NotFound/Fatal を分離し全体消去

この監査に加えて、12 個の固定入力による全精度表参照実装と v2 提案実装の差分試験、および公式入力ベクトル 100 本による通常 API と低メモリ API の差分試験を行った。さらに、保持するビット長と上位ビットを意図的に一致させた場合、NotFound、再試行、Fatal、保護領域、消去の各経路を個別に試験した。ただし、これらは C プログラム全体の意味論や全入力に対する形式証明ではない。対応表、ソースコード位置、機械検査結果は補助資料の <https://github.com/quantumshiro/tinysqisign/tree/main/results/revision-2026-09-04> に置く。

ここで FINISH は、初期化済みの固定精度オブジェクトを終了処理した後、成功、失敗、未発見の別なく作業領域全体を sqisign_secure_clear で消去する共通出口を表す。実装では、長期間生存する格子状態が 77,168 bytes、処理段階ごとの共有領域が 94,912 bytes であり、後者に含まれる候補探索用領域は 14,024 bytes である。したがって、find_uv の型サイズは

$$77,168 + \max(94,912, 14,024) = 172,080 \text{ bytes} \quad (8)$$

となる。

4.3 v3 適応実装における局所的な生存期間に基づくメモリ割当て

公式 v3 の p324_3/m4f を RP2350 向けにコンパイルすると、固定長として報告される最大の関数スタックフレームは、quat_l1l_dual_reduce_ideal の 34,816 bytes であった。この関数には、順に実行される処理の間で生存期間が重ならない領域が二組ある。

第一の領域共有では、格子簡約状態 fp_ldl_t と、簡約後に初めて構築する 4×4 整数行列 Aout を、一つの型付き union へ置く。fp_ldl_t へのポインタは簡約ルーチンへ渡されるため、コンパイラが自動的に同じスタック位置を使うとは限らない。C の型配置で生存期間が重ならないことを明示し、この再利用をリンク済みバイナリへ反映させる。

第二の領域共有では、ct_mk_result_t 内の逆変換行列 Ainv_total から Aout を導出した後、不要となった Ainv_total の領域を基底出力用 Bout として再利用する。同じ構造体内のグラム行列 G と変換 T は、後続のノルム計算まで生存するため共有しない。Ainv_total と Bout の領域サイズが等しいことは Static_assert で検査する。

第三の領域共有は、署名本体 `protocols_sign` に適用する。コミットメントイデアルは `ideal_skchall_aux` の構築と最大公約数検査を終えた後には参照されない。一方、チャレンジ同種写像の出力積は、その後に初めて書き込まれる。この二つを `protocols_sign_phase_t` へ配置し、`p324_3` における同関数のスタックフレームを 20,008 bytes から 19,272 bytes へ減らす。コミットメントイデアルの最終参照が出力積の最初の書込みより前にあることは、ソースコード位置を記録した生存期間確認表で検査する。

以上の三変更を二つの関数へ適用した構成が 2 関数を変更した実装である。アルゴリズム、分岐条件、乱数消費、公開 API は変更せず、局所変数の所有と配置だけを変更する。したがって、`v2` のビット長と上位ビットの組のようなデータ表現の変更ではなく、`equations (1) and (3)` を二つの関数へ適用した例である。1 関数を変更した実装は、実機での反復試験とバイナリ配置試験に用いる。生存期間に基づくメモリ割当てを二つの関数へ転用できるかという評価には、2 関数を変更した実装を用いる。

4.4 生存期間の注釈から配置を生成する手順

生存期間の条件から型付き配置を生成するため、各オブジェクトに `phases` (使用する処理段階)、`size` (大きさ)、`alignment` (アラインメント)、`secret` (秘密性)、`pointer_escape` (ポインタ保持の有無)、`clear_on` (消去時点) を与える注釈形式を用いた。生成器は、処理段階の集合が重なるオブジェクト間に干渉辺を作る。次に、アラインメント制約を満たす先頭適合方式でオフセットを割り当てる。出力は、JSON 形式の配置、干渉グラフ、型付き C の `union` と `struct`、および `_Static_assert` である。

`v2` の `find_uv` 注釈では、77,168 byte の長期格子状態と、三つの処理段階用オブジェクトとの干渉を表した。後三者をオフセット 77,168 へ共有配置し、合計 172,080 byte の配置を生成した。生成したヘッダを Cortex-M33/RADIX32 の実際の型とともにコンパイルし、手動実装した `find_uv_workspace_t` の大きさとオフセットに一致することを確認した。さらに、C ヘッダにある 4 個の物理オブジェクトと、ソースコード中で要素または処理段階全体へ直接アクセスする 18 形式を列挙した。注釈のない要素または直接アクセスがあれば検査を失敗させる。

自動化するのは、与えられた注釈からの干渉グラフ生成、配置、および直接アクセスの網羅性検査である。処理段階の意味、生存期間、間接的な別名参照、ポインタの保持、すべての失敗経路における消去、関数ポインタの呼出し先、リンク時最適化後の呼出しグラフ、および未確定値が外部出力へ渡らないことは、C ソースコードから自動推論しない。このため、配置の妥当性は注釈の完全性に依存する。生存期間確認表、動的なカナリア値検査、消去試験、および ELF 監査による別経路の検査が必要である。

5 実装と評価方法

提案手法を評価するには、整数の表現幅と物理的なメモリ配置を固定し、比較する実装の構築条件を揃える必要がある。機能試験では出力の一致を調べ、メモリ検査では領域共有、消去、および予約量への適合を調べる。

5.1 整数表現とメモリ配置

実装は Compact Quaternion Algorithms の公開実装を出発点とし、SQIsign v2.0.1 Level I と、有限体・曲線層の RADIX32 表現に固定した。RADIX32 は有限体要素の基数表現であり、四元数整数の語幅ではない。四元数層の `ibz_t` は、1665 bit の絶対値上界に符号 bit を加え、64 bit 語の境界へ切り上げた `uint64_t[27]` である。これは 1728 bit の符号付き格納領域であり、1 整数当たり 216 bytes を要する。乗算、除算、最大公約数、平方根、モジュラー指数演算を含む多倍長演算は、この固定配列と専用作業領域上で行う。有限体・曲線側にも既存の固定幅実装を用いる。

この選択により、各整数オブジェクトの容量とアラインメントはコンパイル時に定まり、`equations (1) and (3)` の配置へ直接含まれる。ただし、固定幅であることは演算回数が固定であることを意味しない。現在の使用語数の走査、比較、ユークリッド法、指数演算は可変時間である。本稿は、この整数処理を定数時間とは主張しな

い。v2 提案実装の SQIsign ソースコードはコミット 71099e0827d3f0a3b3c705d2eda592c401e0d57d、RP2350 ファームウェアはコミット dc3289add3213cc7671f9943dfaa3bac770b2709 に固定した。

GMP 6.3.0 の公式マニュアルによれば、mpz_t は整数の大きさと確保済みデータへのポインタを持つ。容量が不足すると領域を追加確保し、既定では malloc、realloc、free を用いる [26]。したがって、通常の GMP は多倍長整数の語を一般ヒープへ確保・再確保する。この方式は、動的メモリ確保関数を含まないベアメタル用コードと、コンパイル時に確定する SRAM 予約合計という本稿の要件を満たさない。mini-GMP は依存関係を小さくできるが、本実験で用いた同梱版も、mpz_t ごとに可変長の語配列を確保する。したがって、未改変の mini-GMP をリンクするだけではメモリ所有の問題は解決しない。

機能試験では、公式ソースコードのコミット dd133d7aca576c361a270c8e6434832535b42ecc にある Level-I 実装について、整数処理だけを切り替えた。システム GMP、未改変の mini-GMP、および消去機能を加えた静的プール版 mini-GMP は、いずれも公式 100 ベクトルの既知解テストに合格した。一方、固定した LP64 ホスト入力集合で Sign を実行すると、未改変 mini-GMP が同時に要求した多倍長整数の語領域は最大 67,168 bytes であったが、同時に生存する領域数は最大 4,830 個だった。16 byte アラインメントの単純な先頭適合型プール割当てでは、100 経路すべてを処理できる最小プールが 317,696 bytes であり、317,680 bytes では枯渇した。この値は、メモリ割当て方式、ABI、および入力集合に依存する測定値である。mini-GMP 固有の下界でも、Cortex-M33 上の上界でもない。このプール量には、公式実装のスタック、その他の書換え可能な状態、および MSP を含めていない。したがって、v2 提案実装のファームウェア全体の値である 321,408 bytes とは直接比較しない。

同じ公式ホストソースコードでは、未改変 mini-GMP のシステム GMP に対する実行時間中央値の比が、KeyGen で 1.3621、Sign で 1.4497 であった。解放時に消去する単純なプール割当ては、要求量を記録する mini-GMP に対して KeyGen で 4.9441 倍、Sign で 4.2316 倍の時間を要した。ただし、これは今回実装したメモリ割当て処理の費用であり、mini-GMP 一般の費用ではない。個別演算では mini-GMP が最大公約数と逆元で速く、固定幅版が今回測定した乗算、除算、平方根で速かった。Arm GNU 15.2.1 が生成したオブジェクトコードのうち、未使用セクション除去前の命令領域は、-Os で固定幅版が 7,962 bytes、mini-GMP 側が 18,566 bytes であった。固定幅版を選んだ主因は速度ではなく、次の性質を全到達経路について監査できることである。

1. 全到達経路でヒープを使用せず、操作用領域の予約量をリンク前に確定できる。
2. 数千個の可変長領域について、断片化、管理情報、解放順を別途解析する必要がない。
3. 操作終了時に共有領域全体を一括消去し、解放済みの語領域に消去漏れがないか検査できる。
4. 確保サイズ、確保回数、再配置先アドレスという追加の入力依存トレースを導入しない。

固定幅版にも値に依存する制御フローとメモリアドレスが残る。したがって、最後の項目は、動的メモリ確保に由来する追加の漏洩面を除くという限定的な意味である。容量制限付き mini-GMP と専用領域割当ても設計候補になり得る。しかし、すべての再試行経路に必要なプール上界、32 bit 実機上の総 SRAM、消去、実行サイクル、および漏洩検査を評価していないため、本稿の比較対象には含めない。

v2 提案実装の操作用アリーナ本体は 172,080 bytes であり、前後の保護領域を含む操作用共有領域は 172,208 bytes である。主 SRAM バンクには、操作用共有領域、131,072 byte の PSP 予約、およびその他 9,936 bytes を配置する。リンク時に確保する主 SRAM バンクの合計は 313,216 bytes である。別の補助バンクにある 8,192 bytes を MSP 用に予約する。したがって、SRAM 予約量の合計は

$$313,216 + 8,192 = 321,408 \text{ bytes} \quad (9)$$

であり、211,072 bytes を未予約のまま残す。

v3 では、公式ソースコードのコミット 6d017708db403bf83977fa70770fc4f7f9e9ff21 を無改変の対照実装とする。1 関数を変更した実装はコミット 9293313fb58de4c5ce9dd27a5a9fde0058766c79、2 関数を変更した実装はコミット 874658c64aa2e20f53b1f4d696144723d558ed5c に固定する。2 関数を変更した実装が変更するのは l1l_dim4.c と sign.c だけである。三つの公式パラメータ集合について、生成元コミットと生成済み pqm4 ソースコードのディレクトリ木ダイジェストを対応付けた。RP2350 用の 2 関数を変更した実装は、Pico SDK

2.3.0、Arm GNU Toolchain 15.2.1、Release 構成、150 MHz、1 コアという条件で、ファームウェアのコミット 5df7acfb2e32019305094546939bdc50bbf71e00 から構築した。再構成用パッチと各ダイジェストは公開補助資料に含める。

公式 v3 は、大きな局所状態を主としてスタックに置く。このため、v2 提案実装の操作用アリーナ本体と v3 の PSP 使用量の実測値は同じ量ではない。v3 ファームウェアでは、アプリケーション用の PSP 領域を 131,072 bytes 予約する。各操作の直前に既知のパターンを書き込み、操作後に連続して上書きされた深さを測る。割込み用の MSP 領域は 8,192 bytes とする。

公式 v3 と v3 適応実装では、リンク済み ELF のヒープ用セクションはいずれも 0 byte である。ただし、診断用の `fprintf/abort` 経路を介して `newlib` の動的メモリ確保関連シンボルが残る。また、GCC のスタック使用量出力には 19 件の動的フレーム記録がある。したがって、v3 適応実装について主張するのは、成功した KeyGen、Sign、Verify 経路でヒープ使用を観測しなかったことと、対象関数の局所スタックフレームを削減したことだけである。v2 提案実装とは異なり、動的メモリ確保関連シンボルと動的フレームをリンク後到達範囲からすべて除いたとは主張しない。

5.2 比較条件と反復設計

評価項目を次の五群に分けた。

1. **機能同値性**: 参照経路との公開鍵、秘密鍵、署名付きメッセージ、乱数生成器の事後状態のバイト一致、正当署名の受理、および改変署名の拒否。
2. **メモリ**: 型の `sizeof`、リンカマップ、ELF セクション、ヒープ使用方針、PSP/MSP 使用量の実測値、および未予約 SRAM。
3. **移植性**: RADIX64/RADIX32、Level I/III/V の関連試験、Arm 向けコンパイル、C の別名参照規則、および可変長配列・`alloca` の禁止。
4. **性能**: ホスト計算機上で実行順を反転した反復試験と、RP2350 実機上の各操作の経過時間。両環境の結果を混同しない。
5. **入力依存性**: 固定入力とランダム入力の実行時間検査、制御フロー・実効アドレス列の再現性、および局所的なモジュラー指数演算の実行時間診断。

v2 提案実装、公式 v3、および v3 適応実装は、固定したコミットから独立に構築し、実装間でオブジェクトファイルを共有しない。比較マニフェストは、各実装のコミット、ELF、実機記録、および各測定量の定義を SHA-256 で対応付ける。

公式 v3 と v3 適応実装は、同じ基準コミット、パラメータ集合、コンパイラオプションを使う。比較対象とするソースコードの差分は、1 関数を変更した実装では一ファイル、2 関数を変更した実装では二ファイルに限定する。この版内比較を、局所的な領域共有の効果推定に用いる。v2 と v3 の間では、パラメータ、アルゴリズム、整数表現、およびメモリ所有方式が同時に変わる。このため、版間の数値は参考値として記述するだけにとどめる。特に、v2 の明示的アリーナと v3 のスタック使用量の実測値を、一つの RAM 使用量として減算しない。

v2 実機試験では、AES-256 CTR-DRBG を用いる決定的な試験用乱数生成器を使用した。ホスト参照と同じ乱数種、メッセージ、期待ダイジェストを用いる。この乱数源は差分検査用であり、製品向けのエントロピー源ではない。クロック、ツールチェーン、リンカ配置、バイナリのハッシュ、実機記録のハッシュをマニフェストへ記録した。UF2 形式の同じファームウェアイメージを二回起動し、KeyGen、Sign、Verify を直列に実行して、各操作後に前後の保護領域と全領域の消去を検査した。これは一つの入力を反復する試験であり、入力分布を広げる試験ではない。

v2 の差分適合試験では、保存済みの NIST-v2 Level-I 公式入力ベクトル 100 本を用いた。入力ファイルの SHA-256 は 81ff60e3...d6ebc7e であり、完全な値は補助資料に記録した。同じコミットから通常 API と `SQISIGN_LOWMEM_ONLY` API を別々に構築し、両方へ同じ入力を与えた。低メモリ側は通常の入口を呼ばず、呼出

し側所有の作業領域を用いる KeyGen、Sign、Open、Verify だけを実行する。各ベクトルについて、正当な署名付きメッセージの復元、署名の 1 bit 改変の拒否、前後の保護領域、および操作後の消去を検査した。通常 API は一回、低メモリ API は別々に起動したプロセスで二回実行した。三つの応答列は一致したが、同じ入力集合と同じコミットの通常経路を共有するため、独立した期待値ではない。また、旧公式応答を期待値とする既知解テストでもない。

v3 実機試験では、公式既知解テストの第 0 ベクトルをファームウェアへ埋め込み、公開鍵、秘密鍵、署名付きメッセージ、復元メッセージをバイト単位で照合した。1 関数を変更した実装は、実行順を反転した 5 組、合計 10 回の起動時測定で公式版と比較した。2 関数を変更した実装は、第 0 ベクトルによる KeyGen、Sign、Verify を一回測定した。報告する測定は、未コミットの変更を含まないソースコードとファームウェアに固定した。バイナリ、実機記録、差分パッチの SHA-256 を公開補助資料に記録した。

ホスト計算機では、公式版と 2 関数を変更した実装を別々に構築した。p324_3、p500_27、p664_17 のすべてについて、API 試験、方式自己試験、および公式応答 100 本を処理した。異なるパラメータと入力の組は、3 集合×100 本の 300 組である。各組を公式版と 2 関数を変更した実装で一回ずつ実行したため、合計 600 回検査した。Arm 用スタックフレームの監査も同じ 6 構成を対象とし、版間でオブジェクトファイルを共有しない。

単一ベクトルの反復とは別に、公式応答の第 0-9 ベクトルを一回の起動で順に処理するファームウェアも構築した。公式版と 1 関数を変更した実装のそれぞれについて、暗号ライブラリの直前へ 0 個または 512 個の Thumb 無操作 (NOP) 命令を置く二つの配置を用いる。後者では、crypto_sign_keypair、crypto_sign、crypto_sign_open の開始アドレスが正確に 1,024 bytes 移動する。この試験では、正当な KeyGen、Sign、Verify の出力一致に加えて、署名先頭の 1 bit を反転した改変署名を各ベクトルで拒否することも確認する。

5.3 機能とメモリ安全性の検証

測定対象のファームウェアでは、Level I/RADIX32 の KeyGen、Sign、Verify から到達するプロジェクト内の翻訳単位を 52 個に固定した。ELF 監査では、動的メモリ確保関数、GMP、システム乱数生成器、旧大容量スタック経路、および未解決シンボルが存在しないことを検査した。また、配置済みのセクションが内部 SRAM または XIP フラッシュの外にないことも検査した。ヒープの予約量は 0 byte である。操作用共有領域の前後には保護領域を置き、各操作後に作業領域全体が 0 であることを確認する。

この方法は、ソースコードに malloc が見えないことだけを根拠にしない。ライブラリ内部の動的メモリ確保、削除されずに残った旧経路、暗黙の可変長配列、およびリンカスクリプト上の未計上領域まで、リンク済み ELF で検査する。

領域共有の安全性を追跡できるように、主要 11 領域について生存期間確認表を作成した。各行には、オブジェクト名、型、Arm ABI 上のサイズ・アラインメント・オフセット、構築地点、最終参照地点、同時使用を禁止する相手、呼出し先へのポインタ渡しと保持の有無、正常・失敗・再試行経路での消去地点、および対応する静的・動的検査を記録する。表 4 はその要約であり、全項目を含む CSV と検査結果は補助資料で公開する。

表 4 v2 で共有する主要領域の生存期間確認表 (要約)

オブジェクト (型)	サイズ [B]	アラインメント [B]	同時使用を禁止する相手	終了・検査
操作用共有領域	172,080	8	KeyGen/Sign/Verify を逐次化	各操作後に全体を 0 走査、前後に保護領域
KeyGen 共有領域	172,080	8	find_uv と離散対数	全終了時に消去、呼出し先の再試行条件、静的表明
Sign 共有領域	172,080	8	7 個の主要処理段階	全終了時に消去、保護領域、100 入力の差分試験
find_uv 格子状態	77,168	8	候補探索と同時に使うため共有しない	構造体の連結、オフセット表明、ASan/UBSan
find_uv 段階共有領域	94,912	8	格子乗算・候補探索・固定次数処理	一方向遷移、Fatal/NotFound を含む全出口で消去
Verify 共有領域	15,428	4	偶数次数同種写像と theta を逐次化	ラッパーの全出口で消去、改変署名試験

ポインタが想定した生存期間を越えて保持されないことを調べるため、対象ソースコードにある大域変数または静的変数への明示的な保存を機械的に走査した。操作用共有領域へのポインタが同期呼出しの

外へ保持される例は見つからなかった。ただし、これはすべての別名参照を検出できる解析ではない。特に Sign の外側の再試行では、各回に 172,080 bytes 全体を消去しない。実行中の呼出し先による終了処理・消去と、次の処理段階による初期化・上書きに依存する。操作用共有領域全体の消去を直接確認したのは、公開作業領域 API の最終出口である。この限定を含む CSV は <https://github.com/quantumshiro/tinysqisign/blob/main/results/revision-2026-09-04/lifetime-certificate-v2.csv>、検査スクリプトは `scripts/check_lifetime_certificate.py` である。

表 5 v2 提案実装に対する主な検証項目

検証項目	判定条件	結果
ビット長と上位ビットの組の衝突	2^{100} と $2^{100} + 1$ が全精度比較を通り正順序	PASS
固定入力	PK/SK/署名付きメッセージが 12 入力すべてでバイト一致	12/12
公式入力の差分適合試験	通常 API と低メモリ API がバイト一致、別プロセスで 2 反復	100/100 × 2
メモリ安全性検査	ASan および UBSan の対象試験で警告なし	PASS
基数・Level	RADIX64/RADIX32 および Level I/III/V の対象試験	PASS
Arm ABI	候補・処理段階・操作領域が規定バイト数と一致	PASS
スタック方針	対象コードで VLA/ <code>alloca</code> と動的フレーム記録が 0	PASS
RP2350 上の全操作	同一 UF2・同一入力の 2 回の起動で全操作成功、保護領域・消去が正常	2/2
Verify の乱数非使用	Verify 前後で乱数生成器の状態が不変	PASS

二回の低メモリ API 試行と一回の通常 API 試行が生成した三つの応答は、すべてバイト単位で一致した。SHA-256 は 1d86d15c5d2bfbef99f17634496086a3ab80f0e848d34d3e9409d75f3019dd68 であった。観測した 100 入力では、作業領域の所有方式を変えたことによる出力差を検出しなかった。ただし、Compact 固定精度版の KeyGen は旧 GMP 実装と乱数消費列が異なるため、保存済みの旧 NIST-v2 応答を期待値には使えない。本稿が主張するのは、公式入力 100 本を用いた、同じ v2 提案アルゴリズム内の通常経路と呼出し側所有経路の差分適合である。旧 GMP 応答との既知解一致ではない。

v3 では公式 p324_3 の既知解を基準とし、無改変実装と v3 適応実装の双方へ同じ検証を適用した。表 6 の実機既知解テストは、自己生成した署名を同じ実装で検証するだけの試験ではない。公式ベクトルに含まれる公開鍵、秘密鍵、署名付きメッセージ、復元メッセージとのバイト一致を要求する。

表 6 SQIsign v3.0 p324_3 の機能検証

検証項目	v3 公式	v3 適応実装
ホスト NIST API 試験	PASS	PASS
ホスト自己試験	PASS	PASS
ホスト公式既知解テスト	PASS	PASS
RP2350 公式既知解テスト	PASS	PASS
改変署名の拒否	PASS	PASS

操作用アリーナ本体、前後の保護領域を含む操作用共有領域、リンク時に確保する主 SRAM バンク、PSP/MSP 予約、およびスタック使用量の実測値を別の量として記録する。スタック使用量は、既知のパターンで事前に埋めた領域が実行中に書き換えられた範囲から測定する（塗りつぶし法）。この実測値は最悪時スタック上界ではない。未予約領域を算出するときは、実測値ではなく、PSP と MSP の予約全体を総 SRAM から差し引く。

6 評価結果

v2 では、アリーナの削減がファームウェア全体の SRAM 予約量へどう反映されるかを、スタック、フラッシュ、実行時間と併せて評価する。v3 では局所的な領域共有の効果を示し、別に構築した固定長配列版でスタック上界

を解析する。最後に、両版に残る入力依存性を評価する。

6.1 RAM・スタック・フラッシュ・実行時間のトレードオフ

表 7 に、全精度表実装から v2 提案実装への差分を示す。全精度ノルム二行を削除したことで、候補探索用領域は 14,024 bytes まで縮小した。しかし、処理段階ごとの共有領域は 94,912 bytes 残る。これは、既存の ML2 再試行用領域が新たに最大の要素となるためである。また、77,168 byte の格子状態は候補探索と同時に生存するため、候補探索用領域とは共有していない。局所配列の大きさだけでなく、同時に生存する領域の関係を調べなければ、この最大要素の移動を説明できない。

表 7 ビット長と上位ビットの組による Cortex-M33/RADIX32 のアリーナ使用量の変化

項目	全精度表 [B]	v2 提案 [B]	差分 [B]
常駐する全精度ノルム 2 行	269,568	0	-269,568
候補探索用領域	275,840	14,024	-261,816
処理段階ごとの共有領域	275,840	94,912	-180,928
操作作用アリーナ全体	353,008	172,080	-180,928

操作作用アリーナ全体の削減率は

$$\frac{353,008 - 172,080}{353,008} = 0.51253229 \quad (10)$$

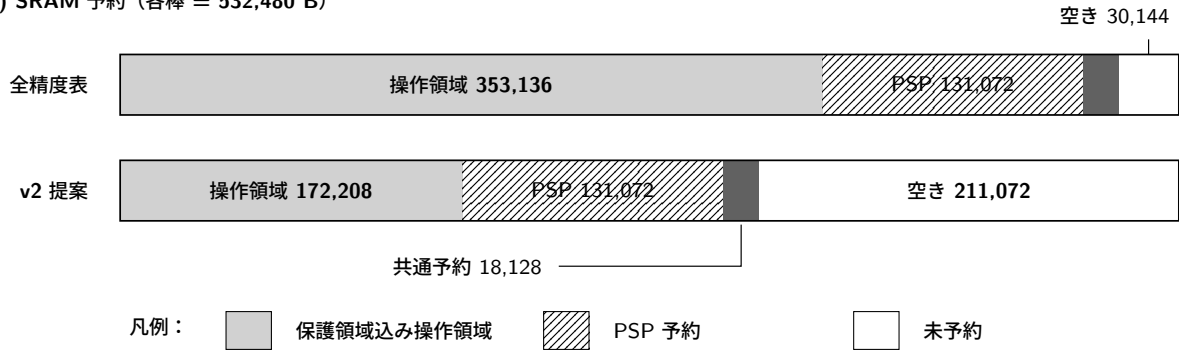
であり、51.2532% である。

表 8 に、v2 提案実装をリンクした結果を示す。全精度表実装から v2 提案実装への 180,928 byte の削減は、そのまま SRAM 予約量の合計の削減と未予約 SRAM の増加に反映された。したがって、ヒープ上のデータをスタックへ移しただけの結果ではない。v2 提案実装のバイナリイメージは 288,384 bytes であり、全精度表実装より 880 bytes 大きい。この増加は、全精度値の再生成と比較を行うコードによるものである。

表 8 v2 提案実装を RP2350 上で動かすために予約したメモリの内訳

項目	v2 提案 [B]	備考
オンチップ SRAM 総量	532,480	主バンク + 補助バンク
操作作用アリーナ本体	172,080	KeyGen・Sign・Verify の共有領域
保護領域込み操作作用共有領域	172,208	主バンク
PSP 予約	131,072	主バンク
その他の主バンク領域	9,936	データ・実行時状態等
主バンクのリンク時使用量	313,216	上記の合計
MSP 予約	8,192	補助バンク
SRAM 予約合計	321,408	全精度表比 -180,928
未予約 SRAM	211,072	全精度表比 +180,928
ヒープ	0	動的メモリ確保シンボルなし
.text	166,092	XIP フラッシュ
.rodata	116,232	XIP フラッシュ
.data	5,984	フラッシュから RAM へ読み込み
.bss	306,960	RAM、予約を含む
バイナリイメージ	288,384	全精度表比 +880

(a) SRAM 予約 (各棒 = 532,480 B)



(b) XIP 用バイナリイメージ

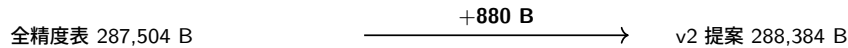


図 4 全精度表実装と v2 提案実装の RP2350 SRAM 予約量。両方の棒は、オンチップ SRAM 総量 532,480 bytes を同じ尺度で表す。操作用共有領域には、前後の保護領域 128 bytes を含む。「共通予約」18,128 bytes は、MSP 予約 8,192 bytes とその他の主 SRAM バンク領域 9,936 bytes の和である。PSP と MSP は実測したスタック使用量の実測値ではなく予約全体を示し、白い部分だけが未予約である。右下のバイナリイメージ差分は、SRAM の棒には含まない。

PSP には 131,072 bytes を予約した。同じ UF2 と決定的入力を用いた二回の起動では、KeyGen で 91,980 bytes、Sign で 120,452 bytes、Verify で 20,768 bytes のスタック使用量の実測値を得た。二回の値は一致した。最も深い Sign でも、観測値と予約量の差は 10,620 bytes であった。MSP の観測値も二回で一致し、8,192 byte の予約に対して保守的に見積もったスタック使用量の実測値は 2,396 bytes であった。

二回の実測値だけでは、入力によって変わる実行経路の上界にならない。このため、測定済み ELF の .text セクションとバイト一致する再構築版を別に解析した。126 個のスタック使用量ファイルにある 1154 件の記録には、動的フレームがなかった。リンク後の KeyGen、Sign、Verify から到達する関数について、スタック情報が欠落する関数と解決できない間接呼出しを 0 件まで調べた。二つの再帰処理では、各子呼出しで入力長が半減するというソースコード上の条件と最大長 248 から、同時に存在するフレームを高々 9 個とした。ソースコードのダイジェスト、生成定数、間接呼出しの解決結果、分岐中継コード、手書きアセンブリまたは newlib のフレーム情報が変わった場合は、検査を停止する。

再帰処理をまとめた呼出しグラフの最長経路に、Armv8-M のセキュア状態における最大例外進入分 212 bytes を加えた結果を表 9 に示す。三操作すべてが PSP 予約内に収まり、最小の余裕は Sign の 10,408 bytes である。実測値とソフトウェア呼出し分の上界が一致したことから、解析対象 ELF と計測経路の対応を確認した。

表 9 固定した v2 ファームウェアイメージの PSP 解析。PSP 上界は、ソフトウェア呼出し分の上界に最大例外進入分 212 bytes を加えた値である。予約余裕は、131,072 byte の PSP 予約との差である。

操作	実機観測 [B]	ソフトウェア呼出し [B]	PSP 上界 [B]	予約余裕 [B]
KeyGen	91,980	91,980	92,192	38,880
Sign	120,452	120,452	120,664	10,408
Verify	20,768	20,768	20,980	110,092

この解析が対象とするのは、固定ファームウェアイメージにリンクされた KeyGen、Sign、Verify のスレッドモード (Thread mode) 経路と、PSP に積まれる一回の最大例外進入分である。割込み処理のソフトウェアフレーム、有効な割込み要求の集合、割込みの入れ子、および例外進入時点で使われている MSP 量は含まない。したがって、リンク後に到達可能な各操作の PSP 経路が予約内に収まるとは述べられるが、割込みを含むプログラム全体の最悪時スタック上界ではない。未予約量 211,072 bytes の算出でも、PSP と MSP の実測値ではなく予約全体を控除する。

表 10 は、一回目の起動で得た経過時間を示す。サイクル数は、操作中のクロックが 150 MHz で一定であるとして、経過時間から換算した値である。性能カウンタから直接取得した値ではない。二回目の起動では、KeyGen、Sign、Verify がそれぞれ 2,698.150324、7,611.257377、0.814356 s であった。一回目との差は、それぞれ 295、-1150、15 μ s である。これは同じ決定的経路の再現性を示すだけであり、入力分布に対する性能分布ではない。KeyGen は約 45 分、Sign は約 127 分を要した。本実装はメモリ内に収められることの実証であり、リアルタイム署名器ではない。

表 10 RP2350 上の決定的な KeyGen、Sign、Verify 実行（一回目の起動。二回目の PSP 使用量の実測値は同一）

操作	時間 [s]	150 MHz 換算サイクル数	PSP 使用量の実測値 [B]
KeyGen	2,698.150029	404,722,504,350	91,980
Sign	7,611.258527	1,141,688,779,050	120,452
Verify	0.814341	122,151,150	20,768

全精度表実装と v2 提案実装の実行順を入れ替えるホスト試験を二回行った。各試験は 15 個の乱数種について二回ずつ両実装を測定した、30 組の対応のある測定からなる。v2 提案実装の全精度表実装に対する実行時間中央値の比は、KeyGen で 1.003695 と 1.006464、Sign で 0.999481 と 0.998980 であった。ホスト計算機上の KeyGen では 1% 未満の増加が見られたが、Sign の比は二回とも 1 を下回った。ただし、これらは有限の測定集合の記述統計であり、一般的な高速化や同等性を示す検定ではない。実機上の単一観測を比べると、KeyGen は 1.000704、Sign は 1.037312、Verify は 1.000593 であった。しかし、実機上の反復測定分布がないため、Sign が一般に 3.7% 遅くなるとは結論しない。

v2 提案実装は全精度表実装に対し、操作用 RAM を 180,928 bytes 削減した。一方、局所スタックフレームは 48-64 bytes、フラッシュ使用量は 880 bytes 増加した。ホスト計算機上の KeyGen では小さな再計算時間が見られたが、Sign では実行時間増加を確認できなかった。実機上の絶対実行時間は長く、実行時間の増加率を推定できる反復測定もない。このため、v2 提案実装がすべての指標で優れるとは位置付けない。本実装は、スタックとフラッシュの小さな増加を受け入れて、操作用 RAM を大幅に削減する構成である。

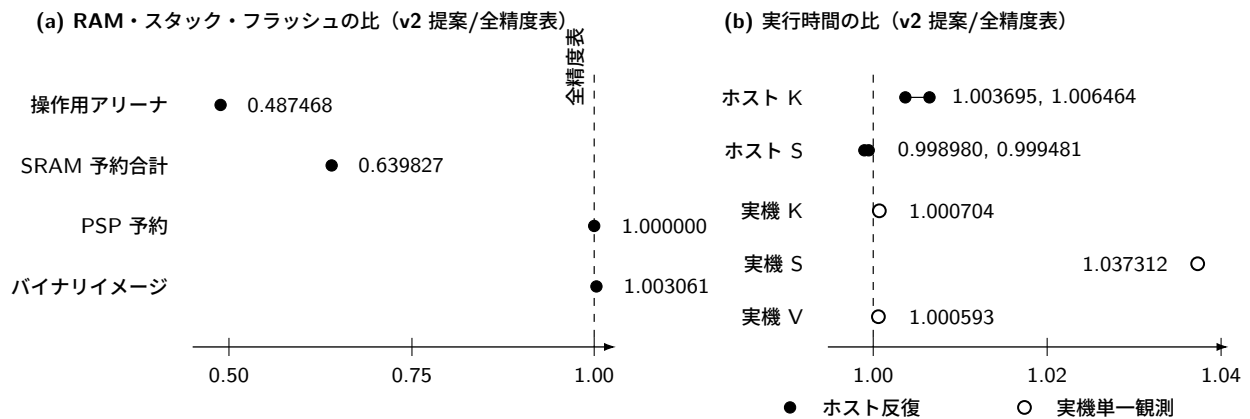


図 5 全精度表実装を 1 とした v2 提案実装の RAM、スタック予約、フラッシュ、および実行時間の比。メモリ量はリンク結果と型サイズから得た値である。ホスト計算機上の実行時間は、実行順を反転した二つの 30 組の測定を別々の点で示す。実機上の実行時間は各一回の観測なので白抜き点とした。測定条件が異なる点は結んでいない。PSP 予約量は変わらないが、局所スタックフレームは 48-64 bytes 増加した。

6.2 SQIsign v3 への適用結果

1 関数を変更した実装は、`quat_l11_dual_reduce_ideal` だけに二組の領域共有を適用する。表 11 は、同じ公式 v3 基準コミットから構築した無改変実装との比較を示す。両実装は、ホスト計算機上でそれぞれ公式応答 100 本を処理した。1 関数を変更した実装は同関数のスタックフレームを 34,816 bytes から 30,888 bytes へ削減した。その差 3,928 bytes は、5 回すべてで Sign の PSP 使用量の実測値の減少量と一致した。この反復試験は、関数の局所スタックフレーム削減が実機のスタック使用量へ反映されるかを調べる。二つの関数へ転用した結果は、後述する 2 関数を変更した実装で評価する。

表 11 RP2350 上の公式 SQIsign v3.0 と 1 関数を変更した実装の比較。時間列は各 5 回の中央値、差分列は 5 組の個別比率の中央値である。

項目	v3 公式	1 関数変更	差分
最大固定スタックフレーム [B]	34,816	30,888	−3,928 (−11.282%)
KeyGen PSP 使用量の実測値 [B]	56,972	56,972	0
Sign PSP 使用量の実測値 [B]	101,060	97,132	−3,928 (−3.8868%)
Verify PSP 使用量の実測値 [B]	37,444	37,444	0
暗号ライブラリの命令領域 [B]	204,157	203,945	−212 (−0.1038%)
KeyGen 時間 [s]	2.263654	2.269166	+0.2416%
Sign 時間 [s]	7.152690	7.182418	+0.4190%
Verify 時間 [s]	0.822142	0.820780	−0.1713%

KeyGen と Verify の PSP 使用量の実測値が変わらないことは、1 関数を変更した実装が Sign だけを変更したものと整合する。三操作の PSP 使用量の実測値は、公式版と 1 関数を変更した実装のいずれでも 5 回の反復で一定だった。Sign 時間の対ごとの増加率は 0.4112–0.4204% で、中央値は 0.4190% だった。この値は、当該バイナリ、基板、クロック、および決定的な既知解ベクトルに対する観測である。入力分布に対する値でも、2 関数を変更した実装の実行時間増加率でもない。

2 関数を変更した実装では、`protocols_sign` の第三の領域共有を加えた。表 12 の RP2350 測定は、未コミットの変更を含まないソースコードとファームウェアに対応付けた。第 0 ベクトルの KeyGen、Sign、Verify は公式応答と一致した。Sign の PSP 使用量の実測値は、公式版の 101,060 bytes から 96,396 bytes へ、合計 4,664 bytes (4.6151%) 減少した。1 関数を変更した実装から追加で減少した 736 bytes は、`protocols_sign` のスタックフレーム削減量と一致する。2 関数を変更した実装の実機測定は一回だけなので、経過時間は性能比較に用いない。

表 12 RP2350 上の公式 v3 と 2 関数を変更した実装の p324_3 比較。関数スタックフレームは Arm オブジェクトファイルから得た静的値である。PSP 使用量の実測値は、公式版では 5 回の中央値、2 関数を変更した実装では一回の観測値である。

項目	公式 v3	2 関数変更	差分
<code>quat_l11_dual_reduce_ideal</code> フレーム [B]	34,816	30,888	−3,928
<code>protocols_sign</code> フレーム [B]	20,008	19,272	−736
KeyGen PSP 使用量の実測値 [B]	56,972	56,972	0
Sign PSP 使用量の実測値 [B]	101,060	96,396	−4,664 (−4.6151%)
Verify PSP 使用量の実測値 [B]	37,444	37,444	0

一つのパラメータ集合だけに依存する変更でないことを調べるため、公式版と 2 関数を変更した実装を三つの公式パラメータ集合すべてで生成した。各構成は、API 試験、方式自己試験、公式応答 100 本を通過した。これは、300 種類のパラメータと入力の組を二実装で検査した、合計 600 回の実行である。表 13 のとおり、二つの関数のスタックフレームは六つの比較すべてで減少した。この結果は、局所的な領域共有を三つのパラメータ集合へ適用できることを示す。ただし、p500_27 と p664_17 について、ファームウェア全体の SRAM 量または実機上のス

タック使用量の実測値を示すものではない。

表 13 三つの公式パラメータ集合における 2 関数を変更した実装の Arm Cortex-M33 用スタックフレーム。

パラメータ	関数	公式 [B]	2 関数を変更した実装 [B]	削減
p324_3	quat_l1l1_dual_reduce_ideal	34,816	30,888	-3,928 (11.2822%)
p324_3	protocols_sign	20,008	19,272	-736 (3.6785%)
p500_27	quat_l1l1_dual_reduce_ideal	46,088	40,904	-5,184 (11.2480%)
p500_27	protocols_sign	27,304	26,328	-976 (3.5746%)
p664_17	quat_l1l1_dual_reduce_ideal	63,016	55,904	-7,112 (11.2860%)
p664_17	protocols_sign	36,728	35,392	-1,336 (3.6376%)

複数入力試験では、ファームウェアのコミット 76f88dcc...、公式ソースコードのコミット 6d017708...、1 関数を変更した実装のコミット 9293313f... から、四つのファームウェアイメージを生成した。すべてのイメージでヒープ用セクションは 0 byte であり、動的メモリ確保関数の入口は三つの強制停止処理だけである。19 件の動的スタック記録は、実装間およびコード配置間で同一だった。表 14 に結果を示す。

表 14 公式応答の 10 入力と二つのコード配置を用いた局所スタック削減の検査。各列は 10 入力 × 2 配置からなる。配置間の PSP 比較は、KeyGen、Sign、Verify、および改変署名の拒否について各 10 入力、合計 40 対を対象とする。

項目	公式 v3	1 関数を変更した実装
正当な KeyGen・Sign・Verify	20/20	20/20
改変署名の拒否	20/20	20/20
Sign PSP 使用量の実測値 [B]	101,060	97,132
配置 A/B 間 PSP 不一致	0/40	0/40
Sign 時間の中央値 [s]	7.134887	7.229949
対応時間差の範囲 [%]	—	1.1327–1.5592

40 個の正当な KeyGen、Sign、Verify 経路と、40 個の改変署名拒否はすべて成功した。各操作の PSP 使用量の実測値は、同じ実装・入力に対する配置 A と B の 80 比較すべてで一致した。1 関数を変更した実装の Sign 削減量も、20 観測すべてで 3,928 bytes だった。したがって、観測した削減は、少なくともこの 10 入力と二配置の範囲では、一つの既知解入力または一つのリンクアドレスだけに依存しない。

一方、同じ複数ベクトル用ファームウェアにおける Sign 時間差は 1.1327–1.5592%、中央値は 1.3323% であり、単一ベクトル用ファームウェアの 0.4190% とは一致しない。したがって、1 関数を変更した実装の一般的な実行時間増加を単一の百分率で主張しない。二つのコード配置間における入力別 Sign 時間の Pearson 相関は、公式版で 0.999999867、1 関数を変更した実装で 0.999999912 だった。入力間の最大時間と最小時間の比は、それぞれ 1.474635、1.468729 であった。ただし、この 10 入力では乱数種、鍵、メッセージが同時に変わる。この試験結果を、固定鍵の秘密情報に由来する漏洩へ帰属させることはできない。

6.3 固定長配列版によるスタック上界の解析

19 件の動的スタック記録を一件ずつ調べ、p324_3/RADIX32 から定まる配列上界を生成ソースコードへ埋め込む固定長配列版を実装した。配列上界を超えた場合は、NDEBUG を指定しても消えない強制停止処理を実行する。-Werror=vla -Werror=alloca を指定して再構築した結果、18 件が固定長のスタックフレームとなった。残る mp_div_unsigned 一件はインライン展開または削除され、コンパイラが報告する動的局所フレームは 0 件となった。ホスト計算機では公式応答 100 本、RP2350 ではコミットに固定したファームウェアイメージの第 0 ベクトルを検査した。ELF では、ヒープ用セクションが 0 byte であることと、三つの動的メモリ確保入口がすべて強制停止処理であることを確認した。

表 15 公式 v3 と固定長配列版の比較。PSP 使用量の実測値は、同じ第 0 ベクトルと同じ基板を用いた別の測定系列から得た。実行時間の比較には用いない。

項目	公式 v3	固定長配列版	差分
動的局所フレーム記録	19	0	-19
暗号ライブラリの命令領域 [B]	204,153	206,297	+2,144
KeyGen PSP 使用量の実測値 [B]	56,972	62,096	+5,124
Sign PSP 使用量の実測値 [B]	101,060	101,060	0
Verify PSP 使用量の実測値 [B]	37,444	40,252	+2,808
ホスト上の公式応答	-	100/100	-

固定長配列版はスタックフレームの大きさを静的に確定できるようにしたが、Sign の PSP 使用量の実測値は減らず、KeyGen と Verify の値は増えた。この結果は、可変長配列を固定長配列へ置き換えるだけで実測スタック量も減る、という予想を支持しない。一方、PSP 予約内に収まるかどうかは実測したスタック使用量の実測値とは別に調べる必要がある。このため、固定長配列版を用いて、リンク結果に固有のソフトウェア呼出し分の PSP 上界を求め、ハードウェア例外進入分を加えた。

まず、正常終了時には到達しない newlib 診断経路の `fprintf` と `abort` を、明示的な強制停止処理へ置き換えた。暗号ライブラリは `-fstack-usage -Werror=vla -Werror=alloca` を指定して再構築した。次に、リンク済み逆アセンブリから BL/BLX 命令、別シンボルへの末尾呼出し、および分岐中継コードを解決し、すべての `.su` 記録と照合した。コンパイラがスタック情報を出力しない固定アセンブリと newlib 補助関数については、SP 減算命令、最大保存量、命令断片、および関数本体の SHA-256 を対応付けた手動記録を用いた。関数内の条件分岐を関数呼出しと誤認しないように区別した。シンボル境界を越える条件分岐 3 件は、いずれも DCP 浮動小数点ラッパーの続きであり、対応する手動スタックフレームへ含めた。その結果、KeyGen、Sign、Verify から到達する関数について、スタック情報の欠落と未解決の間接呼出し先はともに 0 件となった。

呼出しグラフ上の再帰を含む強連結成分は、KeyGen と Sign で 3 個、Verify で 0 個である。離散対数の再帰では、各子呼出しで入力長が半分になる。この構成の最大入力長 198 に対し、同時に存在するスタックフレームを高々 9 個とした。Burnikel-Ziegler 除算では、一回の再帰ごとに多倍長整数の語数が半減する。RADIX32 における最大 60 語に対し、高々 11 個とした。ソースコード上の減少条件または生成定数が変わった場合は、解析を停止する。各再帰成分をこの深さ上界で重み付けし、再帰成分をまとめた呼出しグラフの最長経路を求めると、表 16 を得る。

表 16 固定長配列版について、リンク結果から求めた PSP 上界。ソフトウェア呼出し分に最大例外進入分を加え、131,072 byte の PSP 予約との差を予約余裕とした。

操作根	保守的 PSP 上界 [B]	予約余裕 [B]
KeyGen	108,300	22,772
Sign	127,932	3,140
Verify	40,468	90,604

RP2350 ファームウェアは Armv8-M のセキュア状態で動作する `rp2350-arm-s` 構成である。セキュア状態のソフトウェアでは、浮動小数点演算器の状態が有効なときに非セキュア例外へ進入すると、ハードウェア例外フレームが最大 52 語になる。8 byte アラインメントのため、さらに 1 語が加わる場合がある [27]。そこで、ソフトウェア呼出し分の上界へ一律 212 bytes を加えた。リンク済み ELF 全体で MSR PSP を実行するのは、操作作用呼出しへ入る中継処理の設定命令と復元命令だけである。ハンドラモード (Handler mode) では MSP を使う。このため、操作中に PSP へ積まれる最初の最大例外進入分を上界へ含める。一方、割込み処理のソフトウェアフレームと割込みの入れ子は、別途評価すべき MSP 使用量として残す。

同じコミットに固定した固定長配列版のファームウェアを実機で再測定し、公式第 0 ベクトルについて KeyGen、Sign、Verify の出力一致と最終的な PASS を確認した。観測した PSP 使用量の実測値は、それぞれ 62,096、101,060、

40,252 bytes であり、すべて上記の上界以下だった。特に Verify では、ソフトウェア呼出し分の上界 40,256 bytes に対して 40,252 bytes を観測した。最大例外進入分を加えた上界との差は 216 bytes であった。この近さは解析値と実測値が対応することを示すが、別のバイナリや別の入力に対する余裕を意味しない。

三操作の上界はすべて予約内に収まるが、Sign の余裕は 3,140 bytes に限られる。この解析は、固定コミット、p324_3/RADIX32、現在のコンパイル・リンク結果に対する保守的な PSP 上界である。C およびアセンブリプログラムの意味論を形式検証したものではない。割込み処理側には未解決の間接コールバックが残る。また、有効な割込み要求の集合、割込み優先度、入れ子、および例外進入時に使われている MSP 量も評価していない。したがって、リンク済みの KeyGen、Sign、Verify におけるソフトウェア呼出しと最大例外進入分を含む PSP 上界は主張するが、プログラム全体の最悪時スタック上界とは主張しない。

版間の絶対時間も参考値として記録する。同じ RP2350 を 150 MHz で動作させたとき、v2 提案実装の一回目の起動における KeyGen、Sign、Verify は、2,698.150029、7,611.258527、0.814341 s であった。同じ UF2 を用いた二回目は、2,698.150324、7,611.257377、0.814356 s であった。これは同じ入力に対する起動間の再現性を示すが、入力分布に対する性能標本ではない。v3 公式実装の 5 回の中央値は、2.263654、7.152690、0.822142 s であった。v3 の KeyGen と Sign は大幅に短い、パラメータ、アルゴリズム、整数処理、および実装成熟度が同時に異なる。この差を、一つの省メモリ変換による速度向上率とは解釈しない。また、v2 提案実装は 172,080 byte の明示的アリーナを PSP とは別に持つ。両版の PSP 使用量の実測値だけから、総 SRAM 使用量の優劣を導かない。

6.4 サイドチャネル評価

詳細な攻撃面の追跡は、主として v2 提案実装を対象とする。v3 については、公式応答の 10 入力を二つのコード配置で測る実行時間診断、メッセージと署名乱数を固定して 10 鍵を比較する実行時間検査、ホスト計算機上の制御フロー・実効アドレス診断、および同じ固定条件における RP2350 上の実行時間検査を行った。これらの試験では、鍵ごとに異なる結果を再現できた。ただし、秘密鍵データには対応する公開鍵が含まれるため、観測差を秘密部分だけに帰属できない。公式 v3 仕様も、提出実装全体が完全な定数時間実装ではないと明記する [5]。本研究は電力・電磁波波形の取得と鍵回復を評価していない。したがって、メモリ削減や既知解との一致からサイドチャネル耐性を導くことはできない。

固定したアリーナのオフセットは、動的メモリ確保に由来するアドレス変動を除く。しかし、演算回数やアクセスする値を秘密から独立にはしない。v2 提案実装には、二つのビット長と上位ビットの組が一致する回数、探索を再実行する回数、逆元の存在判定、候補の棄却、および最初の成功時に終了する処理が残る。いずれも入力に依存する。さらに、元の SQIsign 署名経路には、多倍長整数の使用語数走査、比較の早期終了、反復回数が変わるユークリッド法、および秘密に由来する指数を用いるモジュラー指数演算がある。

従って、固定されたメモリ配置だけではサイドチャネル安全性を導けず、RAM 削減後の実装を独立に評価する必要がある。

v2 提案実装について、固定入力群とランダム入力群を各 64 標本測定し、実行順を反転して二回試験した。一次統計 t_1 に加えて、各群の中心値からの偏差を二乗し、その値へ同じ Welch 検定を適用する二次統計 t_2 を求めた。結果を表 17 に示す。この二次統計は、高次の非特定 t 検定で用いられる中心化モーメントによる前処理に対応する [28]。一般的な検出閾値 $|t| > 4.5$ [28] に対し、二回とも t_2 の絶対値が閾値を超えた。同じ入力を比較する対照試験は安定し、二回の試験間における実行時間の Pearson 相関は 0.999929、Spearman 相関は 0.999771 であった。各群 64 標本の小規模な予備検査なので、閾値未満であっても非漏洩の証拠にはできない。一方、実行順を反転した二回の試験で同じ向きの閾値超過を観測した。この結果は、当該条件における実行時間分布の差を示す。

表 17 v2 提案実装の Sign について、ホスト計算機上で固定入力とランダム入力の実行時間を比較した結果

試行	t_1	t_2	判定
A	3.570	−5.410	二次統計が閾値超過
B (順序反転)	3.576	−5.374	同じ向きの閾値超過

ビット長と上位ビットの組の比較回数、保持した情報が一致した回数、探索を再実行した回数と Sign 時間との Pearson 相関は、絶対値がおおむね 0.217 以下であった。したがって、今回測定した指標からは、ノルムの部分保持と再計算が既存の支配的な漏洩源だとは示されなかった。しかし、相関が弱いことは非漏洩の証明ではない。v2 提案実装をサイドチャネル安全性の改善と評価する根拠にもならない。

v3 については、メッセージ、署名乱数、および鍵を格納するアドレスを固定して実行時間を検査した。公式 p324_3 既知解から 10 鍵を選び、各鍵を同じバッファへコピーした。33 byte のメッセージと 48 byte の署名用乱数種を固定し、各鍵を 30 回ずつ、異なる二つのランダム化順序で実行した。Darwin/arm64 のホスト計算機上で、公式版と 1 関数を変更した実装を合わせて 1,200 個の署名を測定し、すべての署名を対応する公開鍵で検証した。両実装とも、二回の試験間における鍵別実行時間中央値の順位は Spearman 相関 0.987879 以上で一致した。最速鍵と最遅鍵の実行時間中央値の差は、全鍵の中央値の 52.6313–53.5166% であった。したがって、このホスト条件では鍵に対応する実行時間差が再現する。ただし、この結果は、差を秘密部分だけへ帰属できること、物理漏洩、または鍵回復を示さない。

同じ仮説を RP2350 でも、測定前に定めた判定規則で検査した。10 鍵を同じ固定アドレスへコピーし、XIP キャッシュを各計測直前に無効化した。メッセージと署名乱数を固定し、鍵順序を変えた二回の試験を各鍵について 5 回ずつ反復した。公式版と 1 関数を変更した実装を合わせて 200 個の署名を取得した。すべての署名が検証に成功することに加えて、署名ダイジェスト、カナリア値、ファームウェアおよびソースコードのダイジェストを確認した。判定規則は、各実装について、二回の試験間における鍵別中央値順位の Spearman 相関が 0.8 以上であることを要求する。また、各試験における最速鍵と最遅鍵の中央値の差が、全鍵の中央値の 1% 以上、かつ鍵内の中央絶対偏差 (MAD) の中央値の 10 倍以上であることを要求する。両実装の Spearman 相関は少なくとも 1.000000 であり、鍵間の差は 50.8419–51.3752% だった。したがって、両実装とも判定規則を満たした。Sign の PSP 使用量の実測値は、100 標本すべてで公式版が 101,060 bytes、1 関数を変更した実装が 97,132 bytes であり、3,928 byte の差も再現した。ファームウェアの実行順は公式版、1 関数を変更した実装の順に固定したため、版間の実行時間差を一般的な性能比較には用いない。この一基板の結果は、RP2350 上で鍵ごとの実行時間差が反復して観測されたことを示す。ただし、公開成分と秘密成分の分離、RP2350 上の制御フロー・アドレストレース、アナログ電力・電磁波漏洩、鍵回復、または耐性を示さない。

同じ入力設計について、Apple Clang の SanitizerCoverage を用い、`crypto_sign` 実行中の制御フロー辺識別子と、読み込み・書き込み命令が使用した実効アドレスを二つの独立したプロセスで記録した。既知解表中の鍵ごとに格納先が異なるという自明な差を除くため、各秘密鍵を計測前に同じ固定アドレスのバッファへコピーした。公式版と 1 関数を変更した実装を合わせて同じ鍵同士を比較した 8 組では、事象数と二種類の列ダイジェストがすべて一致した。他の 9 鍵を鍵 0 と比較する 36 組では、制御フローの事象数またはダイジェスト、および実効アドレスの事象数またはダイジェストが、それぞれ 36/36 組、36/36 組で相違した。記録列は最大約 7000 万個の制御フロー辺と 3 億 5800 万個のメモリアクセスからなる。先頭 200 万事象だけを保存し、列全体は事象数と二つの 64 bit ダイジェストで比較した。このため、同じ鍵同士の一致を形式的な同値性とはみなさない。一方、異なる鍵を比較したすべての組では事象数自体も異なったため、鍵ごとの構造差をダイジェストの衝突確率に依存せず確認できる。この試験は、鍵中の公開成分と秘密成分を分離せず、ホスト計算機用コードだけを観測する。したがって、秘密漏洩、RP2350 用コード、物理波形、または攻撃成立の証拠ではない。

別の二回のホスト計測では、Sign 実行中の制御フロー辺列と、対象となる読み込み・書き込み命令の実効アドレス列を記録した。同じ乱数種同士を比較した対照試験では両方の列が一致した。一方、固定乱数種と異なる署名乱数を比較したすべての組で、両方の列が再現可能に相違した。最初の差分から到達した処理には、多倍長整数の使用語数走査、最上位語の比較、およびランダムイデアル標本化が含まれた。

この結果は、コンパイル済みの署名処理全体に、秘密から到達できる可変制御があることを示す。ただし、変化させた直接入力に署名乱数であり、最初の差分だけから長期秘密鍵が回復できるとは主張しない。

Mukherjee らは、Cornacchia 処理中のモジュラー指数演算で使われる、秘密に由来する一時指数を単純電力解析で回復し、秘密鍵回復へ接続する攻撃を報告した [13]。本実装で 12 個の固定乱数種を用いて Sign を実行すると、Sign → RepresentInteger → Cornacchia → `sqrt_mod_p` の経路を 361 回観測した。そのうち 190 回

は、既報攻撃の指数関係に直接対応する法の分類だった。

RP2350 上でモジュラー指数演算だけを測定したところ、指数の Hamming 重みと経過時間の Pearson 相関は 0.999860、回帰直線の傾きは 1 bit の増加当たり約 4.004 ms だった。この結果は実チップ上の強い可変時間性を示す。ただし、GPIO 信号で区切った局所処理の時間計測であり、アナログ電力・電磁波波形または署名処理全体からの鍵回復実験ではない。

指数処理順の固定、乗算回数の固定、および保護した平方根処理の Sign への統合という三つの局所対策版を測定した。しかし、いずれにも値に依存する差または上位処理の可変回数が残った。入力集合、標本数、タイマー、分散、および各実装の時間比は補助資料の experiments/sca/README.md に示す。したがって、これらの変更は署名処理全体のサイドチャネル耐性を支持しない。耐性を評価するには、署名処理全体の定数時間化またはマスキングに加え、独立に取得した電力・電磁波波形が必要である。また、既報攻撃を検出できる測定系であることも確認する必要がある。

本研究で得た検出結果

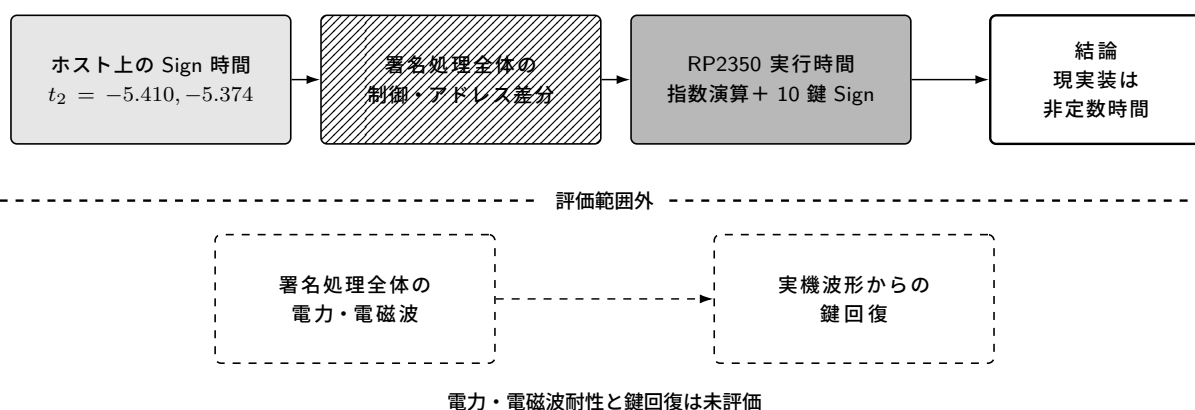


図 6 サイドチャネル評価の範囲。実線枠は本研究で得た検出結果、破線枠は実施していない実験を表す。現実装の可変時間性は検出したが、署名処理全体の電力・電磁波への耐性や、鍵を回復できるかどうかは判定していない。

7 考察と限界

7.1 手法の適用範囲と組み込み上の意味

二つの版に共通して有効だったのは、各オブジェクトの生存期間と干渉関係を明示し、その結果を型付き領域の配置へ反映する手順である。v2 提案実装は、この手順をプロトコル全体の操作共有領域へ適用した。v3 の 2 関数を変更した実装は、再設計後の二つの関数へ同じ手順を適用した。その結果、三つのパラメータ集合における六つのスタックフレーム比較すべてで使用量が減少した。p324_3 の実機試験では、二つの関数で得た削減が Sign の PSP 使用量の実測値へ反映され、公式版との差は 4,664 bytes となった。この結果は、生存期間に基づくメモリ割当てを v2 の find_uv 以外の処理へも適用できることを示す。

一方、ビット長と上位ビットの組、1665 bit の整数幅、172,080 byte のアリーナ、および動的メモリ確保関数と可変長配列を除去した 52 翻訳単位の到達範囲は、v2 に固有である。v3 には対象となる find_uv 候補表が存在せず、2 関数を変更した実装には 19 件の動的スタック記録が残る。このため、v2 の具体的なメモリ配置を v3 へそのまま移植できたとは結論しない。転用できたのは、生存期間に基づいて共有可能性を判断する問題設定と検査手順である。共有した領域と得られた保証は、版ごとに異なる。

本研究で得た具体的なメモリ量は、各版の Level I 相当パラメータと RP2350 に依存する。一方、次の条件を満たす実装には、同じ考え方を適用できる可能性がある。

1. 大きな中間値を、入力または小さな再生成情報から再構成できる。
2. 処理を複数の段階へ分けられ、段階を越えて作業領域へのポインタを保持しない API を設計できる。
3. パラメータごとに配列上界とアラインメントをコンパイル時に固定できる。
4. 代表値だけで順序を決められない場合に、全精度比較へ戻れる。

ノルム比較の要点は、ビット長または上位 bit が異なる場合に比較を確定し、両方が同じ場合は必ず元の全精度値へ戻ることである。この方法はノルムに限らず、小さな入力から比較対象を全精度で再構成できる探索や枝刈り処理へ適用できる可能性がある。ただし、データ量の削減を実証したのは v2 のノルムだけである。生存期間に基づくメモリ割当てについては、v2 の操作用共有領域と v3 の二関数・三パラメータ集合で結果を得たが、他の耐量子暗号方式や他のマイクロコントローラでは実証していない。したがって、SQIsign 以外へ適用できるという見通しは、今後検証すべき仮説である。

v2 提案実装では 211,072 bytes が未予約となり、全精度実装の 30,144 bytes に比べて組込み時の余裕が大きくなった。この余裕は、製品用乱数生成器、通信バッファ、割込みスタック、サイドチャネル対策用の追加状態、または別パラメータ集合への拡張に利用できる可能性がある。ただし、未予約であることは、これらをすべて同時に収容できることを示さない。定数時間化またはマスキングを導入した後は、RAM、スタック、フラッシュ、実行時間を改めて測定し、本実装とのトレードオフを評価する必要がある。v3 については、v3 適応実装の局所スタック削減だけを測定しており、同じ定義による SRAM 予約量の合計と未予約 SRAM は算出していない。

7.2 評価の妥当性と残る制約

本結果には次の限界がある。

1. **対象範囲:** 実機測定は RP2350、Arm Cortex-M33、1 コアに限られ、v2 は Level I/RADIX32 だけを対象とする。v32 関数を変更した実装のホスト機能試験と Arm 用スタックフレーム監査は三つのパラメータ集合を含むが、ファームウェア全体の構築と実機測定は p324_3/m4f だけである。結果は本文に示したコミットに対応し、他のマイクロコントローラまたは異なる実装版へ適用できることを示さない。
2. **入力範囲:** v2 の実機測定は、再現用 CTR-DRBG を用いて同じ決定的な KeyGen、Sign、Verify を二回の起動で反復した。v31 関数を変更した実装では、実行順を反転した単一ベクトル 5 組と、10 ベクトルを二つのコード配置で測定した。2 関数を変更した実装の実機測定は第 0 ベクトル一回である。これらの試験から入力分布、基板の個体差、再試行回数が大きい入力、または 2 関数を変更した実装の実行時間分布は得られない。エントロピー源、乱数種の再設定、故障時処理を含む製品用乱数生成器も統合していない。
3. **機能同値性:** v2 では 12 個の固定入力と、公式入力 100 本による通常・低メモリ API の差分を検査した。v3 では 300 種類のパラメータと入力の組を二実装で処理し、合計 600 回の公式応答一致を検査した。いずれも有限の入力集合を用いた回帰試験であり、全入力に対する C プログラムの同値性を証明しない。v2 差分試験の期待値は同じコミットの通常 API であり、旧 GMP 実装の応答ではない。
4. **人手で定める条件:** 注釈から配置を生成するツールと網羅性検査は、宣言済み要素と直接アクセスの注釈漏れを検出する。しかし、処理段階、生存期間、間接的な別名参照、ポインタ保持、および失敗経路での消去は人手で定める。ASan・UBSan、カナリア値、ELF 監査、および公式応答は別経路の誤り検出手段だが、別名参照を含むすべての C プログラム挙動を証明しない。
5. **スタック上界:** v2 の上界は、固定ファームウェアイメージの入力依存スレッドモード経路と、一回の最大例外進入分を扱う。v3 固定長配列版は動的フレーム記録を 0 件にした別のバイナリであり、その上界は 19 件の動的記録が残る 2 関数を変更した実装には適用できない。割込み処理の入口候補となる 4 関数の直接呼出し先では、スタック情報の欠落を検出しなかった。一方、18 地点の間接コールバック、有効な割込み要求の集合、優先度、入れ子、割込み処理のフレーム、および例外進入時の MSP 使用量は評価していない。したがって、いずれも割込みを含むプログラム全体の上界ではない。
6. **安全性範囲:** v2 署名には、入力に依存する分岐、メモリアドレス、再試行、再計算が残る。v3 でも鍵ごとの

実行時間差と制御フロー・メモリアドレス差を観測したが、公開鍵成分と秘密鍵成分が同時に変わるため、差を秘密部分だけに帰属できない。署名処理全体の電力・電磁波波形、独立取得、攻撃再現、マスキング、および RP2350 上の構造トレースは評価していない。したがって、サイドチャネル耐性は示していない。

7. **実用性:** v2 の KeyGen と Sign は、多くの製品用途には長すぎる。未予約の 211,072 bytes は組込み時の余裕であり、製品用乱数生成器、通信、割込み、および対策用状態を同時に収容できることを実証した値ではない。v3 についても、同じ定義による SRAM 予約量の合計は算出していない。

表 18 は、スタックに関して得られた結果と、その結果に含まれない要素を実装ごとに示す。

表 18 スタック評価の範囲。固定長配列版と 2 関数を変更した実装は別のバイナリである。

対象	得られた証拠	含まれない要素
v2 の KeyGen・Sign・Verify 用 PSP	リンク済みの入力依存スレッドモード経路と一回の最大例外進入分の上界	割込み処理、割込みの入れ子、例外進入時の MSP 使用量
v3 固定長配列版の操作用 PSP	リンク済みソフトウェア呼出しと一回の最大例外進入分の上界	2 関数を変更した実装への適用、割込み処理と MSP
v32 関数を変更した実装の操作用 PSP	二関数の局所フレーム差と一回の p324_3 スタック使用量の実測値	動的フレーム 19 件を含む操作経路の上界
割込み側スタック	MSP 予約・スタック使用量の実測値、直接呼出し先のスタック情報監査	有効な割込み要求の集合、間接コールバック、割込みの入れ子、例外進入時の MSP 使用量

8 結論

SQIsign v2.0.1 Level I に対し、型付きの生存期間に基づくメモリ割当てと、ノルムの部分保持および再計算を組み合わせ、元的全精度比較順序を保ったまま操作用アリーナを 353,008 bytes から 172,080 bytes へ削減した。RP2350 ファームウェアは GMP、動的確保、外部 PSRAM を用いない。予約する SRAM の合計は 321,408 bytes であり、211,072 bytes を未予約として残す。公式入力 100 本による差分適合試験、生存期間確認表、ELF 監査、および固定ファームウェアイメージの PSP 解析は、この範囲における機能・メモリ結果を支持する。

SQIsign v3.0 では、同じ配置原理を `quat_111_dual_reduce_ideal` と `protocols_sign` へ適用し、三つのパラメータ集合における六つの Arm 用スタックフレームを削減した。p324_3 の実機試験では、Sign の PSP 使用量の実測値が 101,060 bytes から 96,396 bytes へ減少した。一方、ビット長と上位ビットの組は v2 固有である。また、固定長配列版の PSP 上界は 2 関数を変更した実装や割込み処理用 MSP には適用できない。

v2 と v3 の両方で、入力に依存する実行時間、制御フロー、およびメモリアドレスが残る。v2 の KeyGen と Sign は、実用的な実行時間にも達していない。したがって、本稿の結果は、省メモリ配置とその検査方法に関するものである。定数時間性、物理漏洩への耐性、プログラム全体の最悪時スタック上界、または製品利用可能性は示していない。

成果物と再現性

ソースコード、再構成用パッチ、実験スクリプト、バイナリのダイジェスト、および実機記録を含む補助資料は、<https://github.com/quantumshiro/tinysqisign> で公開している。

参考文献

- [1] G. Alagic, M. Bros, P. Ciadoux, Q. Dang, T. H. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller. *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum*

- Cryptography Standardization Process*. Tech. rep. NIST IR 8610. National Institute of Standards and Technology, May 2026. DOI: 10.6028/NIST.IR.8610. URL: <https://doi.org/10.6028/NIST.IR.8610>.
- [2] National Institute of Standards and Technology. *Round 3 Additional Digital Signature Schemes*. 2026. URL: <https://csrc.nist.gov/projects/pqc-dig-sig/round-3-additional-signatures> (visited on 09/04/2026).
 - [3] L. De Feo, D. Kohel, A. Leroux, C. Petit, and B. Wesolowski. “SQISign: Compact Post-Quantum Signatures from Quaternions and Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2020*. Vol. 12491. Lecture Notes in Computer Science. Springer, 2020, pp. 64–93. DOI: 10.1007/978-3-030-64837-4_3. URL: https://doi.org/10.1007/978-3-030-64837-4_3.
 - [4] SQISign submission team. *SQISign: Algorithm Specifications and Supporting Documentation, Version 2.0.1*. July 2025. URL: <https://sqisign.org/spec/sqisign-20250707.pdf> (visited on 09/04/2026).
 - [5] SQISign submission team. *SQISign: Algorithm Specifications and Supporting Documentation, Version 3.0*. Tech. rep. National Institute of Standards and Technology, Sept. 2026. URL: <https://sqisign.org/spec/sqisign-20260901.pdf> (visited on 09/04/2026).
 - [6] W. Kim, J. Lee, H. Kim, and C. Lee. “SQISign with Fixed-Precision Integer Arithmetic”. In: *Public-Key Cryptography – PKC 2026*. Vol. 16553. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/1649. Springer, 2026, pp. 3–30. DOI: 10.1007/978-3-032-26737-5_1. URL: https://doi.org/10.1007/978-3-032-26737-5_1.
 - [7] W. Kim, C. Lee, and H. Yoo. “Compact Quaternion Algorithms for SQISign”. In: *Cryptology ePrint Archive, Paper 2026/1031* (2026). URL: <https://eprint.iacr.org/2026/1031>.
 - [8] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang. “pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on Microcontrollers”. In: *Cryptology ePrint Archive, Paper 2024/112* (2024). URL: <https://eprint.iacr.org/2024/112>.
 - [9] F. Carvalho Rodrigues, D. Gazzoni Filho, G. Adj, I. A. Canales-Martínez, J. Chávez-Saab, J. López, M. Scott, and F. Rodríguez-Henríquez. “Generation of Fast Finite Field Arithmetic for Cortex-M4 with ECDH and SQISign Applications”. In: *Cryptology ePrint Archive, Paper 2025/1322* (2025). URL: <https://eprint.iacr.org/2025/1322>.
 - [10] M. A. Aardal, G. Adj, A. Alblooshi, D. F. Aranha, I. A. Canales-Martínez, J. Chávez-Saab, D. L. Gazzoni Filho, K. Reijnders, and F. Rodríguez-Henríquez. “Optimized One-Dimensional SQISign Verification on Intel and Cortex-M4”. In: *Cryptology ePrint Archive, Paper 2024/1563* (2024). URL: <https://eprint.iacr.org/2024/1563>.
 - [11] L. De Feo, L.-J. Jian, T.-Y. Wang, and B.-Y. Yang. “SQISign on ARM”. In: *Cryptology ePrint Archive, Paper 2026/394* (2026). URL: <https://eprint.iacr.org/2026/394>.
 - [12] SQISign submission team. *The SQISign Implementation, Version 3.0*. Git tag `nist-v3`; frozen revision 6d017708... Sept. 2026. URL: <https://github.com/SQISign/the-sqisign/tree/nist-v3> (visited on 09/04/2026).
 - [13] A. Mukherjee, M. Czaprynko, D. Jacquemin, P. Kutas, and S. Sinha Roy. “Simple Power Analysis Attack on SQISign”. In: *Progress in Cryptology – AFRICACRYPT 2025*. Vol. 15651. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/830. Springer, 2025, pp. 245–269. DOI: 10.1007/978-3-031-97260-7_12. URL: https://doi.org/10.1007/978-3-031-97260-7_12.
 - [14] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. “Register Allocation via Coloring”. In: *Computer Languages* 6.1 (1981), pp. 47–57. DOI: 10.1016/0096-0551(81)90048-5.

- [15] J. Gergov. “Algorithms for Compile-Time Memory Optimization”. In: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*. ACM/SIAM, 1999, pp. 907–908. DOI: 10.5555/314500.315082.
- [16] G. Borin, M. Corte-Real Santos, J. Komada Eriksen, R. Invernizzi, M. Mula, S. Schaeffler, and F. Vercauteren. “Qlapoti: Simple and Efficient Translation of Quaternion Ideals to Isogenies”. In: *Cryptology ePrint Archive, Paper 2025/1604* (2025). URL: <https://eprint.iacr.org/2025/1604>.
- [17] Y.-F. Lai. “Qlapoti+ and More: Optimizing Isogeny-based Signatures”. In: *Cryptology ePrint Archive, Paper 2026/1640* (2026). URL: <https://eprint.iacr.org/2026/1640>.
- [18] M. Duparc, A. Leroux, and S. Schaeffler. “Qlapoty: Improved Analysis and Efficiency for Quaternionic Ideal to Isogeny Transformation”. In: *Cryptology ePrint Archive, Paper 2026/1700* (2026). URL: <https://eprint.iacr.org/2026/1700>.
- [19] Y. Hu, S. Shen, H. Yang, and W. Wang. “Paving the Way for SQIsign: Toward Efficient Deployment on 32-bit Embedded Devices”. In: *Mathematics* 12.19 (2024), p. 3147. DOI: 10.3390/math12193147. URL: <https://doi.org/10.3390/math12193147>.
- [20] W. Wang, C. Wang, Y. Wu, Q. Xue, J. Zheng, and Y. Zhao. “Exposing SIMD Parallelism in SQIsign: An AVX-512 Implementation”. In: *arXiv preprint arXiv:2608.13948* (2026). DOI: 10.48550/arXiv.2608.13948. URL: <https://arxiv.org/abs/2608.13948>.
- [21] Anchorage Digital. *sqisign-rs: A Pure Rust Implementation of SQIsign v2.0*. Repository release 0.5.0. 2026. URL: <https://github.com/anchorageoss/sqisign-rs> (visited on 09/04/2026).
- [22] F. Kouider, A. Mukherjee, D. Jacquemin, and P. Kutas. “Constant-time Integer Arithmetic for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/832* (2025). URL: <https://eprint.iacr.org/2025/832>.
- [23] O. Hanyecz, A. Karenin, E. Kirshanova, P. Kutas, and S. Schaeffler. “Constant Time Lattice Reduction in Dimension 4 with Application to SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/027* (2025). URL: <https://eprint.iacr.org/2025/027>.
- [24] A. Basso, C. He, D. Jacquemin, F. Kouider, P. Kutas, A. Mukherjee, S. Schaeffler, and S. Sinha Roy. “Constant-time Quaternion Algorithms for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/2192* (2025). URL: <https://eprint.iacr.org/2025/2192>.
- [25] Raspberry Pi Ltd. *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. RP-008373-DS-2. 2025. URL: <https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline> (visited on 09/04/2026).
- [26] T. Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. 2023. URL: <https://gmplib.org/gmp-man-6.3.0.pdf> (visited on 09/04/2026).
- [27] J. Yiu. *How Much Stack Memory Do I Need for My Arm Cortex-M Applications?* Arm. Mar. 2016. URL: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/how-much-stack-memory-do-i-need-for-my-arm-cortex-m-applications> (visited on 09/04/2026).
- [28] T. Schneider and A. Moradi. “Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations”. In: *Journal of Cryptographic Engineering* 6.2 (2016), pp. 85–99. DOI: 10.1007/s13389-016-0120-y. URL: <https://eprint.iacr.org/2015/207>.